

UNIVERSITY OF LJUBLJANA
FACULTY OF COMPUTER AND INFORMATION SCIENCE

Tim Vučina

**A Node-Based Web Editor for
Customizable Audio-Driven
Animations Rendered in HTML
Canvas**

MASTER'S THESIS

THE 2ND CYCLE MASTER'S STUDY PROGRAMME
COMPUTER AND INFORMATION SCIENCE

SUPERVISOR: doc. dr. Aleš Smrdel

Ljubljana, 2025

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Tim Vučina

**Vozliščno zasnovan spletni urejevalnik
za na platnu HTML upodobljene
prilagodljive zvočno vodene animacije**

MAGISTRSKO DELO
MAGISTRSKI ŠTUDIJSKI PROGRAM DRUGE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

MENTOR: doc. dr. Aleš Smrdel

Ljubljana, 2025

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati zaključnega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda zaključnega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

ACKNOWLEDGMENTS

I would like to extend my deepest gratitude to my colleagues at Zeroday, whose mentorship and camaraderie have been instrumental in bringing this work to life. It was through our collaborative projects and daily challenges that I truly learned and honed the web development technologies and techniques that form the foundation of this thesis. Your willingness to share knowledge, to challenge my ideas, and to support my growth as a developer has been invaluable. Thank you for creating an environment where learning thrives and for your unwavering encouragement throughout this journey.

Tim Vučina, 2025

To the signal in the noise.
And to those who taught me how to listen.

*“The point is to set up a system and
see what it does. And the fun is to make a
system that is surprising to you.”*

— Brian Eno

Contents

Abstract

Povzetek

Razširjeni povzetek	i
I Kratek pregled sorodnih del	i
II Predlagana metoda	ii
III Eksperimentalna evaluacija	iii
IV Sklep	iv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Description	2
1.3 Goals and Objectives	2
1.4 Thesis Structure	3
2 Background and Foundational Technologies	5
2.1 Audio Feature Extraction	5
2.2 Web-Based Visualization and Rendering (2D and 3D)	7
3 Related Work	11
3.1 Desktop-Based Professional Tools	11
3.2 Web-Native Creative Platforms	12
3.3 Creative Coding Frameworks	13

4	System Architecture	15
4.1	A Conceptual Model of VizEngine	15
4.2	System Overview	19
4.3	The Layer-Based Compositing System	24
4.4	The Node-Based Animation Engine	30
4.5	The Audio Processing Pipeline	36
4.6	The Declarative Configuration and UI System	41
4.7	The State Management Architecture	46
4.8	Project Persistence and Data Management	51
4.9	The Export Pipeline	54
4.10	The Editor UI and User Experience	59
5	Implementation	65
5.1	Technology Stack	65
5.2	Extensibility and Developer Experience	68
5.3	Source Code Organization	72
5.4	Testing and Quality Assurance	74
6	Evaluation	79
6.1	Evaluation of Expressive Capabilities	79
6.2	Performance Evaluation	88
6.3	Comparative Analysis	95
6.4	Usability Study	98
7	Conclusion	105
7.1	Summary of Contributions	105
7.2	Limitations	107
7.3	Future Work	107
7.4	Final Remarks	110
A	VizEngine Project Resources	111
A.1	Source Code Repository	111
A.2	Live Demonstration	112

CONTENTS

B Profiler Metric Definitions	113
B.1 Core Timing Metrics	113
B.2 Component-Specific Metrics	115

List of used acronyms

acronym	meaning
API	Application Programming Interface
BPM	Beats Per Minute
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DAG	Directed Acyclic Graph
DOM	Document Object Model
DTO	Data Transfer Object
DX	Developer Experience
EDM	Electronic Dance Music
FFT	Fast Fourier Transform
FPS	Frames Per Second
GPGPU	General-Purpose computing on Graphics Processing Units
GPU	Graphics Processing Unit
GTCCs	Gammatone Cepstral Coefficients
HTML	HyperText Markup Language
I/O	Input/Output
LLM	Large Language Model
MFCCs	Mel Frequency Cepstral Coefficients
NPM	Node Package Manager
RMS	Root Mean Square
SDF	Signed Distance Field
SPA	Single-Page Application
UI	User Interface
URL	Uniform Resource Locator
Wasm	WebAssembly
WebGL	Web Graphics Library
ZCR	Zero-Crossing Rate

Abstract

Title: A Node-Based Web Editor for Customizable Audio-Driven Animations
Rendered in HTML Canvas

Existing tools for creating customizable audio-driven animations often present challenges related to complexity, proprietary licensing, or the need for extensive coding. This thesis introduces VizEngine, an open-source, web-based editor designed to offer a more accessible alternative for this task. VizEngine employs a hybrid architecture that integrates a layer-based compositing system with a declarative, node-based animation engine, enabling users to visually construct reactive behaviors by connecting audio analysis outputs to visual parameters. The prototype system was implemented using modern web technologies, primarily React for the user interface, Three.js for 2D/3D rendering via WebGL, and the Web Audio API for audio processing. Evaluation included assessing the system's expressive capabilities through demonstrative compositions and conducting a detailed, cross-platform performance analysis using a custom in-engine profiler. The performance results validate the core architecture's efficiency, indicating that the node engine imposes minimal computational overhead and the system can achieve high average frame rates on different hardware configurations. The analysis also identified specific component rendering costs and main-thread contention, primarily during JavaScript garbage collection, as the main factors limiting stability under heavy load. The primary contribution is a functional prototype and architectural design that demonstrates the feasibility of using standard web technologies to build performant, extensible, and accessible tools for

creating audio-reactive visuals.

Keywords

Audio-Driven Animations, Node-Based Editor, React, Three.js, Web-Based Visualization, Real-Time Rendering, Open-Source Tools

Povzetek

Naslov: Vozliščno zasnovan spletni urejevalnik za na platnu HTML upodobljene prilagodljive zvočno vodene animacije

Obstoječa orodja za ustvarjanje prilagodljivih zvočno vodenih animacij so pogosto zapletena, lastniška ali zahtevajo obsežno programiranje. Magistrsko delo predstavlja VizEngine, spletni urejevalnik, ki zapolnjuje to vrzel. Naš pristop temelji na novi hibridni arhitekturi, ki združuje sistem slojnega kompozitiranja z deklarativnim, vozliščno zasnovanim urejevalnikom animacij. Ta zasnova uporabnikom omogoča vizualno gradnjo kompleksnih odzivnih sistemov. Prototip, implementiran s tehnologijami React, Three.js in Web Audio API, dokazano zmore upodabljati zahtevne 3D-scene v realnem času pri visokih frekvencah osveževanja. Glavni prispevek dela je zmogljivo, razširljivo in odprtokodno ogrodje, ki potrjuje sodobni spletni brskalnik kot zrelo platformo za kreativna orodja.

Ključne besede

Audio-Driven Animations, Node-Based Editor, React, Three.js, Web-Based Visualization, Real-Time Rendering, Open-Source Tools

Razširjeni povzetek

Magistrsko delo obravnava načrtovanje, implementacijo in evalvacijo sistema VizEngine, prototipnega spletnega orodja za ustvarjanje prilagodljivih, zvočno vodenih vizualizacij. Delo neposredno naslavlja vrzel med kompleksno, lastniško programsko opremo za namizne računalnike in programersko zahtevnimi ogroddji za kreativno kodiranje. V ta namen smo razvili dostopno, razširljivo in zmogljivo platformo, ki v celoti temelji na sodobnih spletnih tehnologijah. Sistem temelji na inovativni hibridni arhitekturi, ki združuje paradigmo slojnega kompozitanga in deklarativnega vozliščnega urejevalnika animacij. Cilj dela je bil razviti dostopno orodje, ki bi omogočalo ustvarjanje kompleksnih zvočno vodenih vizualizacij brez potrebe po specializiranem programerskem znanju, hkrati pa zagotavljalo dovolj fleksibilnosti za napredne uporabnike. Ta razširjeni povzetek predstavlja ključne vidike dela: pregled sorodnih rešitev, opis predlagane arhitekture sistema, rezultate evalvacije in končne ugotovitve.

I Kratek pregled sorodnih del

Analiza obstoječih rešitev je ključna za umestitev prispevka tega dela v širši kontekst. Pregledali smo uveljavljena orodja na področju vizualnega programiranja in kreativnega kodiranja. Profesionalna namizna orodja, kot je **TouchDesigner**, ponujajo izjemno zmogljivost in razširljivost, vendar so lastniška in primarno namenjena delu na namizju. Na drugi strani spektra so platforme, kot je **cables.gl**, ki uspešno prenašajo vozliščno paradigmo v brskalnik, vendar delujejo po “freemium” poslovnem modelu in so namenjene

širšemu spektru interaktivne grafike.

Naša analiza je pokazala, da obstaja tržna niša za orodje, ki bi združevalo moč vozliščnega urejanja z dostopnostjo odprtokodne zasnove in bi bilo specifično osredotočeno na **spletno, zvočno vodeno animacijo**. VizEngine je zasnovan z namenom, da zapolni to vrzel kot popolnoma odprtokodna alternativa, ki razvijalcem in umetnikom ponuja transparentno in prosto razširljivo platformo.

II Predlagana metoda

Arhitektura sistema VizEngine je temeljni prispevek tega dela. Zasnovana je modularno, s poudarkom na ločevanju odgovornosti (*ang. separation of concerns*), kar zagotavlja robustnost in razširljivost. Ključni stebri arhitekture so:

Hibridna arhitektura Sistem združuje dve osrednji paradigmi: **slojni kompoziting sistem**, ki uporabnikom omogoča zlaganje in mešanje različnih vizualnih elementov, ter **vozliščni urejevalnik animacij**, ki omogoča dinamično animiranje parametrov teh slojev. Ta kombinacija ponuja tako strukturiran pristop k sestavljanju scene kot tudi izjemno fleksibilnost pri ustvarjanju kompleksnih, proceduralnih animacij.

Upodabljalni cevovod, ki temelji na DOM Ključna inovacija sistema je upodabljalni cevovod, ki temelji na DOM. Namesto tradicionalnega pristopa z enim samim elementom `<canvas>`, vsakemu sloju dodelimo lasten element `<canvas>` v strukturi DOM. Končno sliko nato sestavi brskalnik sam z uporabo standardnih lastnosti CSS, kot sta `opacity` in `mix-blend-mode`. Ta pristop izkorišča strojno pospešeno upodabljanje brskalnika in bistveno poenostavlja logiko upodabljanja.

Deklarativni konfiguracijski in uporabniški vmesnik Namesto ročnega programiranja uporabniških vmesnikov za vsako vizualno komponento

smo razvili deklarativni sistem. Razvijalec definira **konfiguracijsko shemo** parametrov komponente, iz katere sistem samodejno generira interaktivni uporabniški vmesnik. Ta shema služi kot edini vir resnice (*ang. single source of truth*) za validacijo, shranjevanje vrednosti in povezavo z animacijskim sistemom.

Upravljanje stanja in cevovod za obdelavo zvoka Celotno stanje aplikacije upravlja knjižnica **Zustand** po vzorcu več ločenih, domensko specifičnih shramb (*ang. multi-store pattern*). Za obdelavo zvočnega signala smo implementirali **dvopotni zvočni graf**, ki ločuje pot signala za predvajanje od poti za analizo. S tem zagotavljamo visoko kakovost zvoka brez artefaktov in hkrati natančno analizo signala v realnem času.

III Eksperimentalna evaluacija

Evalvacijo prototipa smo izvedli na štirih ključnih področjih, da bi celovito ocenili njegovo uspešnost.

Evalvacija izraznih zmožnosti Pripravili smo tri kompleksne vizualne kompozicije, da bi demonstrirali kreativni potencial sistema. Prva je bila osredotočena na napredno **ekstrakcijo zvočnih značilk**, druga je prikazala zmožnosti **sestavljanja več slojev** (2D, 3D, senčilniki), tretja pa je služila kot stresni test z upodabljanjem kompleksne, navigabilne **3D-scene**. Rezultati so potrdili, da arhitektura omogoča ustvarjanje širokega spektra vizualno bogatih in dinamičnih animacij.

Ocenjevanje zmogljivosti Meritve so pokazale, da na moderni strojni opremi (Apple MacBook Pro M1 Pro) sistem ohranja stabilno povprečno hitrost osveževanja slike pri **58.5 sličic na sekundo (FPS)**, kar je blizu ciljni vrednosti 60 FPS, poraba pomnilnika pa je ostala stabilna. Rezultati potrjujejo, da je izbrana arhitektura tehnično primerna za aplikacije v realnem času.

Primerjalna analiza S primerjavo sistema VizEngine z orodjema TouchDesigner in cables.gl smo potrdili, da prototip uspešno zapolnjuje nišo na trgu. Njegove ključne prednosti so odprtokodnost, spletna dostopnost in specifična osredotočenost na zvočno odzivne animacije.

Študija uporabnosti Izvedli smo kvalitativno študijo uporabnosti s petimi profesionalnimi programerji. Rezultati so pokazali, da je paradigma vizualnega programiranja za tehnično podkovane uporabnike zelo intuitivna, saj so jo hitro povezali z znanimi koncepti reaktivnega programiranja. Hkrati so razkrili priložnosti za izboljšanje uporabniške izkušnje z uvedbo standardnih funkcij, kot je zgodovina razveljavitev (ang. undo).

IV Sklep

V tem delu smo uspešno načrtovali, implementirali in ovrednotili prototipno spletno aplikacijo VizEngine. Glavni prispevki dela so **načrt hibridne arhitekture, delujoč prototip**, ki dokazuje tehnično izvedljivost z uporabo sodobnih spletnih tehnologij, ter **celovita evalvacija**, ki potrjuje izrazni potencial in zmogljivost sistema.

Kljub uspešni izvedbi ima prototip določene omejitve, predvsem enonitno arhitekturo. Kljub temu delo odpira številne smeri za prihodnji razvoj, vključno s prehodom na večnitno izvajanje (Web Workers) in evolucijo v **oblačno platformo za sodelovanje v realnem času**. Zaključimo lahko, da se je brskalnik razvil v zmogljivo platformo za naslednjo generacijo odprtokodnih in sodelovalnih kreativnih orodij.

Chapter 1

Introduction

Audio visualizations are widely used in various fields, from entertainment and music production to data presentation. The creation of compelling and dynamic audio-reactive visuals often requires specialized tools that can be both powerful and intuitive.

1.1 Motivation

Despite this need, a significant gap exists for platforms that are simultaneously accessible to non-programmers, extensible for developers, and native to the web ecosystem. This thesis is motivated by the desire to bridge this gap by providing an **accessible platform** that empowers users, regardless of deep programming knowledge, to design **generative and customizable animations**. Furthermore, by developing an **open-source solution**, this work aims to foster a community around the tool, encouraging collaborative enhancement and the widespread re-usability of created animations and components.

1.2 Problem Description

Despite the demand, accessible and intuitive tools for creating highly customizable, audio-driven animations are scarce. Many existing solutions, such as Resolume [1] and TouchDesigner [2], are proprietary, limiting their accessibility and modification, or are not web-based, restricting their ease of sharing and deployment. Furthermore, tools like TouchDesigner, while powerful, often present a steep learning curve and are designed for live performance installations. Additionally, while open-source alternatives like Processing [3] and p5.js [4] offer creative coding environments, they often require more direct programming effort for complex user interfaces or streamlined web deployment for the specific type of **node-based editor** envisioned here. Some tools also lack the flexibility required by users who wish to design reusable, parameter-driven animations with complex interactions.

1.3 Goals and Objectives

This thesis aims to address this gap by exploring the potential of a node-based visual programming paradigm for creating customizable, audio-driven animations within a web browser environment. The primary goals of this work are:

1. **To investigate the feasibility and effectiveness** of using a **network data graph** approach for defining the relationship between real-time audio analysis and visual parameters. This goal draws inspiration from the demonstrated power of modular, node-based connections in systems like Interstell.AR [5] for music visualization and concepts from tools like Blender’s Sound Nodes [6] in other creative domains.
2. **To design and develop a prototype system (VizEngine)** that embodies this node-based paradigm, specifically tailored for audio-reactivity and web-native deployment.

3. **To demonstrate the integration** of modern web technologies—specifically the **Web Audio API** for audio processing, **HTML Canvas** and **WebGL** (via libraries like Three.js) for 2D/3D rendering, and **React** for the user interface—into a cohesive and functional application for this purpose.
4. **To evaluate the potential** of such a system **through qualitative usability studies and quantitative performance benchmarks**, assessing its ability to provide a higher-level abstraction, empowering users to create sophisticated, generative visuals without requiring deep expertise in low-level signal processing or graphics programming. This includes exploring architectural patterns that support reusable visual **components, layering, and standard blending options**.
5. **To contribute an open-source solution** that fosters accessibility and encourages community collaboration in the creation and sharing of audio-reactive visual content.

Ultimately, this thesis seeks to establish whether a browser-based, node-driven approach can offer a compelling balance of power, accessibility, and flexibility for the creation of dynamic audio visualizations.

1.4 Thesis Structure

This thesis is organized as follows. Chapter 2 introduces the **core principles** of audio-driven visualization, covering fundamental concepts of audio processing and feature extraction relevant to creating reactive visuals. Chapter 3 delves into a review of **related work**, examining existing audio visualization tools, node-based editing paradigms, and relevant web technologies. Chapter 4 outlines the proposed **system architecture**, detailing the core components and their interactions. Chapter 5 presents the **implementation**, detailing how the system architecture is realized in practice. Chapter 6 presents the **evaluation of expressive capabilities**, showcasing examples

of visualizations that can be created with the editor and assessing its usability, performance, and expressive power. Finally, Chapter 7 concludes the thesis by summarizing the key **contributions**, discussing limitations, and suggesting avenues for future research and development.

Chapter 2

Background and Foundational Technologies

To understand the design and implementation of a web-based, audio-driven animation editor, it is essential to first review the established technologies and literature in its two principal areas.

First, the chapter delves into established techniques for **audio feature extraction** (Section 2.1), surveying the algorithms and toolkits used to translate raw audio signals into meaningful data. This forms the analytical basis for any audio-reactive system.

Subsequently, it examines the landscape of **web-based visualization and rendering** (Section 2.2), from low-level browser APIs like WebGL to the high-level libraries such as Three.js and React that make complex, interactive graphics feasible. This review establishes the technical environment in which VizEngine operates and provides context for the implementation choices discussed in Chapter 4.

2.1 Audio Feature Extraction

This section reviews specific algorithms and widely-used techniques for deriving **meaningful descriptors** from audio signals. These are essential for

creating sophisticated audio-reactive systems, and form the analytical core of **VizEngine**.

Key time-domain features include **Root Mean Square (RMS) energy**, which provides a measure of the signal's overall intensity or power, effectively indicating its loudness [7, 8]. Another common time-domain feature is the **Zero-Crossing Rate (ZCR)**, which counts how often the audio signal crosses the zero amplitude level. ZCR is often indicative of the noisiness of a signal or can be a simple discriminant between voiced and unvoiced speech, or percussive versus sustained sounds [7, 8].

Frequency-domain analysis, primarily enabled by the **Fast Fourier Transform (FFT)** [9, 8], reveals the spectral makeup of the signal by decomposing it into its constituent frequencies. From this spectral representation, features like **Spectral Centroid** are derived. The Spectral Centroid represents the center of mass of the spectrum, effectively indicating the "brightness" of a sound; a higher centroid value corresponds to more energy in higher frequencies [10, 8]. **Spectral Flux** quantifies the rate of change of the power spectrum between successive analysis frames, thereby highlighting moments of significant timbral change or the occurrence of new sonic events [10, 8]. Other useful descriptors include **Spectral Rolloff**, which indicates the frequency below which a certain percentage of the total spectral energy lies, and **Spectral Flatness**, which measures the uniformity of the power spectrum. These features collectively offer insights into the timbral characteristics of sound.

For analyses more aligned with human auditory perception, features like **Mel Frequency Cepstral Coefficients (MFCCs)** [11, 12] and **Gamma-tone Cepstral Coefficients (GTCCs)** [13, 14] are prominent. MFCCs, derived from Mel-scaled filterbank energies, are efficient timbre descriptors widely used in music information retrieval. GTCCs similarly use perceptually inspired Gammatone filters and have demonstrated strong performance in various audio analysis tasks. While powerful, the real-time implementation of these more advanced features requires careful optimization.

Detecting rhythmic events, particularly **onsets**, is vital for synchronizing visuals with musical rhythm. Algorithms for **onset detection** typically analyze sudden energy increases or spectral changes, with methods ranging from simple energy-based approaches to more complex multi-band spectral flux analysis.

The **Web Audio API** provides essential browser-based audio processing capabilities. Furthermore, JavaScript libraries such as **Essentia.js** [15, 16] and **Meyda.js** [17] have been pivotal in making a wide array of these features, from basic RMS to MFCCs, readily available for web developers. These libraries significantly simplify the creation of real-time audio-reactive applications by encapsulating complex signal processing tasks. Historically, toolkits like **MIRtoolbox** [18] for Matlab have also been highly influential in the broader field of music information retrieval, pioneering many analysis tools.

The survey by Bonet-Solà and Alsina-Pagès [19] offers a recent and comprehensive comparison of various feature extraction methods and their performance in diverse acoustic environments, providing valuable context for selecting appropriate features.

2.2 Web-Based Visualization and Rendering (2D and 3D)

The rendering of dynamic graphics within a web browser is facilitated by a stack of technologies, ranging from low-level Application Programming Interfaces (APIs) that communicate directly with the system’s graphics hardware, to high-level libraries that provide productive and powerful abstractions. This section reviews these foundational technologies, establishing the technical landscape in which VizEngine operates. We will first examine the native browser APIs for 2D and 3D rendering and then discuss the influential libraries that build upon them.

2.2.1 Foundational Browser APIs

At the core of web-based graphics is the HTML `<canvas>` element. It provides a resolution-dependent bitmap drawing surface within the DOM, but it is the JavaScript contexts obtained from this element that provide the actual drawing capabilities.

The primary and most widely supported context is the **Canvas 2D API**. It is an imperative, stateful API for drawing 2D graphics. A developer issues a sequence of commands to set styles (such as colors and line widths) and then draw shapes, text, or images as shown in listing 1. While straightforward for simple graphics, managing the state for complex, object-oriented scenes can become cumbersome, as the developer is responsible for manually redrawing the entire scene on every frame.

```
1 // Example of the imperative drawing style of the Canvas 2D API.  
2 const ctx = canvas.getContext('2d');  
3 ctx.fillStyle = 'blue';           // Set the state  
4 ctx.fillRect(20, 20, 100, 80); // Issue a drawing command
```

Listing 1: A minimal example of drawing a rectangle using the Canvas 2D API.

For high-performance 2D and 3D graphics, browsers provide the **Web Graphics Library (WebGL)** API [20]. WebGL is a low-level API based on OpenGL ES, which provides a direct interface to the host system’s Graphics Processing Unit (GPU). This enables hardware-accelerated rendering, which is essential for complex scenes and smooth animation. However, this power comes at the cost of significant complexity. A developer using raw WebGL must manually manage shader programs, buffer data, and matrix transformations for camera and object positioning. Due to this high verbosity, WebGL is rarely used directly for application development; instead, it serves as a compilation target or a foundational layer for higher-level libraries.

2.2.2 High-Level Rendering Libraries

To manage the complexity of the native APIs, a rich ecosystem of JavaScript libraries has emerged. These libraries provide abstractions that allow developers to work with higher-level concepts, such as scenes, objects, and components, rather than individual drawing commands or GPU buffers.

For 2D creative coding, **p5.js** [4] is a highly influential library. Inspired by the Processing language [3], p5.js provides a simplified and accessible API for drawing on an HTML canvas. Its primary focus is on education and artistic expression, making it an excellent tool for sketching and learning. However, its global state model and drawing-loop paradigm are less suited for building the kind of complex, component-based editor architecture required by VizEngine.

For 3D graphics, **Three.js** [21] has established itself as the de facto standard library for the web. It provides a comprehensive set of tools that abstract away the complexity of WebGL. Its core abstraction is the **scene graph**—an object-oriented structure composed of scenes, cameras, lights, and meshes (which combine geometry and materials). This allows developers to build and manipulate 3D worlds in a structured and intuitive manner. VizEngine leverages Three.js for its entire 3D rendering pipeline, benefiting from its robust feature set and performance optimizations.

While these libraries manage the state of the visual canvas, a separate concern is managing the state of the editor’s user interface itself. For this, we utilize **React** [22], a declarative UI library. React’s component-based architecture is ideal for building the complex interface of an application like VizEngine, including its panels, forms, and interactive controls. This creates a clear separation of concerns: React manages the state and rendering of the editor’s DOM elements, while libraries like Three.js manage the state and rendering within the `<canvas>` element.

The architectural approach of VizEngine is to deliberately select and combine the strengths of these technologies. It uses React to build its interactive user interface, while providing distinct rendering paths that can

leverage either the directness of the Canvas 2D API for simple 2D components or the power of the Three.js scene graph for complex 3D components.

Chapter 3

Related Work

Having established the foundational technologies in the previous chapter, this chapter reviews existing **integrated systems and editors** that combine audio processing with visual generation. By analyzing the current landscape, from professional desktop applications to web-native platforms and coding frameworks, we can identify the specific niche that VizEngine aims to fill. This review contextualizes the contributions of this thesis by comparing its goals and architecture to established solutions. We categorize these solutions into desktop applications, web-native platforms, and creative coding frameworks.

3.1 Desktop-Based Professional Tools

The most powerful and mature tools for real-time generative visuals are currently desktop applications. The foremost example in this category is **TouchDesigner** [2]. It is a highly versatile environment that has become an industry standard for creating interactive media systems, live event visuals, and installations. Its core is a **node-based paradigm** that is a direct and significant influence on VizEngine. TouchDesigner demonstrates the immense expressive power of connecting operators in a dataflow graph for real-time visual generation. However, it is a proprietary, closed-source application with a steep learning curve. Furthermore, its focus on desktop and installation

contexts means it is not designed for a **web-native workflow** of easy sharing and embedding, a primary goal of our project.

Other professional tools like **Notch** [23] offer similar node-based, real-time graphics capabilities for the live events industry. Historically, visual programming languages like **Max/MSP** [24], particularly with its Jitter extension for video processing, pioneered the use of a node-based paradigm for multimedia art and music, establishing the validity of this approach long before modern real-time graphics became commonplace. These systems validate the power of the node-based paradigm but also highlight the gap for an accessible, open-source, and web-based alternative.

3.2 Web-Native Creative Platforms

In recent years, several powerful creative tools have emerged that operate entirely within the web browser. The most direct analogue to VizEngine’s visual editor is **cables.gl** [25]. It is a web-based platform that provides a node-based interface for creating real-time 3D graphics and interactive web content. It successfully demonstrates that complex visual programming is feasible on the web. While cables.gl is a general-purpose graphics tool, VizEngine is differentiated by its specific and deep focus on **audio-reactivity** as a first-class citizen, with a core library of audio analysis nodes and a workflow tailored to this purpose.

A contrasting web-native approach is found in **Hydra** [26], a popular environment for **live coding**. Hydra allows users to write GLSL shader code in a minimalist, immediate feedback loop, generating complex video feedback and patterns. While also web-based and generative, Hydra’s code-first, imperative paradigm is fundamentally different from the visual, declarative dataflow model of VizEngine’s node editor.

3.3 Creative Coding Frameworks

Creative coding frameworks provide a lower-level, code-centric approach to creating visuals. As discussed previously, libraries like **Processing** [3] and its web-focused successor **p5.js** [4] empower developers with a high degree of control and flexibility. However, they are fundamentally toolkits for programmers. Creating an application with a sophisticated user interface, such as a node editor or a layer stack, requires significant, bespoke development effort on top of the framework’s core rendering functionality. VizEngine aims to provide this higher-level application structure out-of-the-box, allowing users to focus on the creative task rather than the application architecture.

Academic research has also explored this domain. The system described in **Interstell.AR** [5], for example, presents a real-time music visualization system that demonstrates the viability of a modular, audio-driven approach, a paradigm that is also central to the design of VizEngine.

In summary, the landscape of creative visual tools presents a spectrum, from professional-grade desktop applications to flexible but code-intensive frameworks. A distinct gap exists for an open-source, web-native tool that combines the intuitive, generative power of a node-based editor with a specific focus on audio-reactivity. VizEngine is designed to fill this niche, offering a higher-level abstraction than a coding framework while remaining more accessible and web-friendly than proprietary desktop software.

Chapter 4

System Architecture

The creation of compelling, custom audio-reactive visuals has traditionally been bifurcated between inflexible, template-based applications and complex, programming-centric creative coding frameworks [3, 4]. This thesis presents the architecture of **VizEngine**, a system designed to occupy the space between these two extremes. A primary goal of this work is to provide the flexibility of a coding framework through a highly modular, node-based visual paradigm, thereby making dynamic visual creation accessible to a broader audience.

This chapter deconstructs this goal into its core architectural principles by establishing a conceptual model. We begin with the most abstract definition of an audio visualizer and progressively introduce layers of complexity to arrive at the concrete, high-level design of VizEngine prototype.

4.1 A Conceptual Model of VizEngine

Before deconstructing the system’s software components, it is useful to establish a conceptual model that formally defines its function. At its core, any audio visualizer can be described as a system that transforms an audio signal into a sequence of images. We model VizEngine’s architecture through three levels of increasing abstraction, starting from this fundamental premise.

4.1.1 Level 1: The Stateless Transducer

At the most abstract level, the system can be modeled as a pure, stateless function—a transducer that converts one signal type to another. This function, which we denote as V , takes an audio signal A and a specific point in time t as its inputs. Its sole output is a corresponding image frame I . This relationship can be expressed mathematically as:

$$V(A, t) \mapsto I \quad (4.1)$$

The key property of this model is that it is deterministic and free of side effects, a core principle of functional programming [27]. Given the same audio data and the same timestamp, the function V will always produce an identical image frame. This represents the most fundamental form of an audio-visual mapping, as depicted in Figure 4.1.

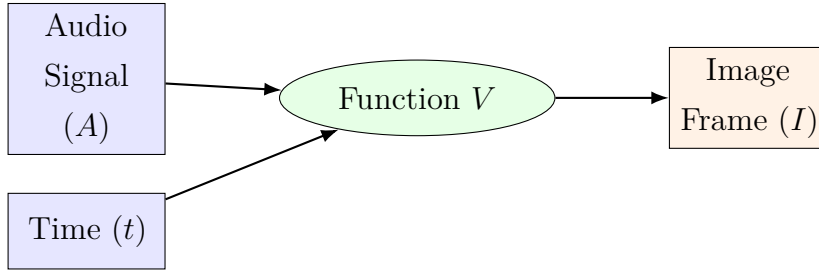


Figure 4.1: A stateless transducer model of an audio visualizer, where the output is a pure function of the audio signal and time.

4.1.2 Level 2: Introducing State

While the stateless model is fundamental, it lacks the flexibility required for a creative tool. To enable user customization, we must introduce the concept of **state**. In our model, we define a state object S that encapsulates the user’s entire project configuration. This includes the chosen visual elements, their parameters, and the animation logic that connects them to the audio.

The visualization function V is now dependent on this state. It transforms

the audio signal based on the rules and configurations defined within S . The function's signature is updated to include this new input:

$$V(A, t, S) \mapsto I \quad (4.2)$$

This revised model, shown in Figure 4.2, moves the system from a simple transducer to an interpreter. It interprets the user's intent, as captured in the state S , to produce the final visual output. This architectural decision to centralize user configuration into a well-defined state object is a common pattern in interactive systems, ensuring predictability and enabling features such as project saving and loading [28].

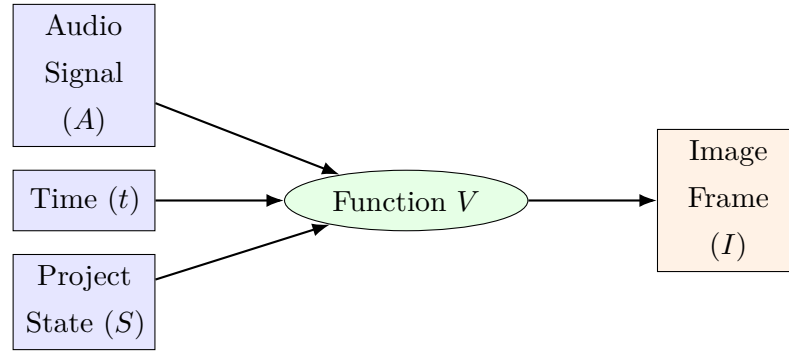


Figure 4.2: A stateful model where a persistent project state (S) modifies the visualization function's output.

4.1.3 Level 3: Decomposing the System Architecture

The third and final level of our conceptual model deconstructs the abstract stateful function into its concrete architectural components. This decomposition forms the high-level blueprint for the VizEngine prototype, as illustrated in Figure 4.3.

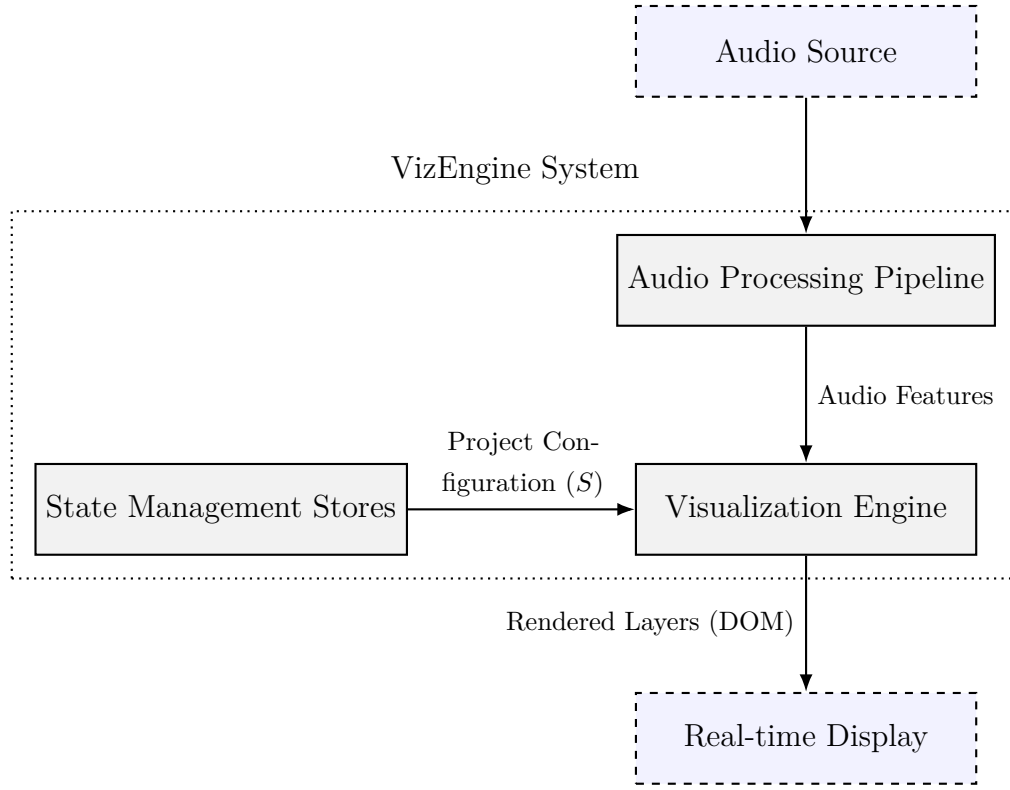


Figure 4.3: High-level decomposition of the VizEngine architecture into its primary subsystems (vertical flow). The Visualization Engine renders to individual layer canvases within the DOM, and the browser handles the final real-time compositing.

The abstract "Project State" (S) is realized as a collection of specialized **State Management** stores. The single "Function" (V) is broken down into a dataflow pipeline comprising the primary subsystems:

- **Audio Processing Pipeline:** This subsystem is responsible for ingesting the raw audio signal and executing a user-defined graph of analysis nodes to extract meaningful features (e.g., RMS energy, spectral flux), as discussed in Section 2.1.
- **Visualization Engine:** This is the core rendering component. It takes the features from the audio pipeline and the configuration from

the state stores to render the visual elements for a given frame. It manages the scene graph, camera, and lighting, leveraging Three.js for 3D rendering [21]. As detailed in Section 4.3.3, it outputs its layers to stacked `<canvas>` elements, delegating the final compositing to the browser.

This separation of concerns is a key architectural decision. It decouples audio analysis from rendering, allowing each subsystem to be developed and optimized independently. The following sections of this chapter will explore the design and inner workings of each of these components in detail.

4.2 System Overview

The VizEngine is a web-based system designed for the creation of dynamic, audio-reactive visualizations. The architecture is founded on three primary functional pillars: an **Audio Processing Pipeline** for signal analysis, a **Visualization Engine** for rendering, and a **Composition Layer** for timing and export. These components work in concert to translate audio data into visual output in real-time. A high-level view of the editor interface, which exposes the controls for these underlying systems, is shown in Figure 4.4.



Figure 4.4: An overview of the VizEngine editor interface, showing the layer stack, preview canvas, and parameter controls.

4.2.1 The Core Architectural Pillars

We designed the system around a clear separation of concerns, which resulted in three distinct, cooperative subsystems. This modularity ensures that each component has a single, well-defined responsibility.

1. **The Audio Processing Pipeline** is responsible for ingesting and analyzing audio signals. It leverages the browser’s native **Web Audio API** to perform real-time frequency and time-domain analysis. Whether the input is a local file or a live audio stream, this pipeline’s sole output is a consistent, frame-by-frame stream of structured audio feature data, which is then made available to the Visualization Engine.
2. **The Visualization Engine** is the central rendering subsystem. It interprets the user’s project state—a collection of visual layers and their associated animation node-graphs—and uses the feature data from the audio pipeline to generate the final image. A key feature of its design is a multi-canvas architecture that supports both 2D and 3D (via Three.js

[21]) rendering paths, compositing the final output from individual layer canvases.

3. **The Master Clock** provides the primary timing mechanism for the entire application. While the **Remotion** library is used to provide a stable, frame-accurate 60 FPS clock that dictates the update cycle, its export capabilities are not used. As detailed in Section 4.9, a custom, offline export pipeline was developed to better support the system’s stateful, generative architecture.

4.2.2 System Synchronization and Data Flow

For the system to function correctly, the audio playback, visual updates, and the internal clock must remain synchronized. We achieve this by anchoring all operations to the frame-based clock provided by the **Master Clock**. Every $\frac{1}{60}$ th of a second, the system performs a single, atomic update cycle.

This design enforces a strictly **unidirectional data flow**, as illustrated in Figure 4.5. Data originates from two sources—the audio signal and the user’s project configuration—and flows through a series of transformations, resulting in a final rendered image for that frame. The absence of reverse data pathways is a deliberate architectural choice that simplifies state management and makes the system’s behavior highly predictable and easier to debug.

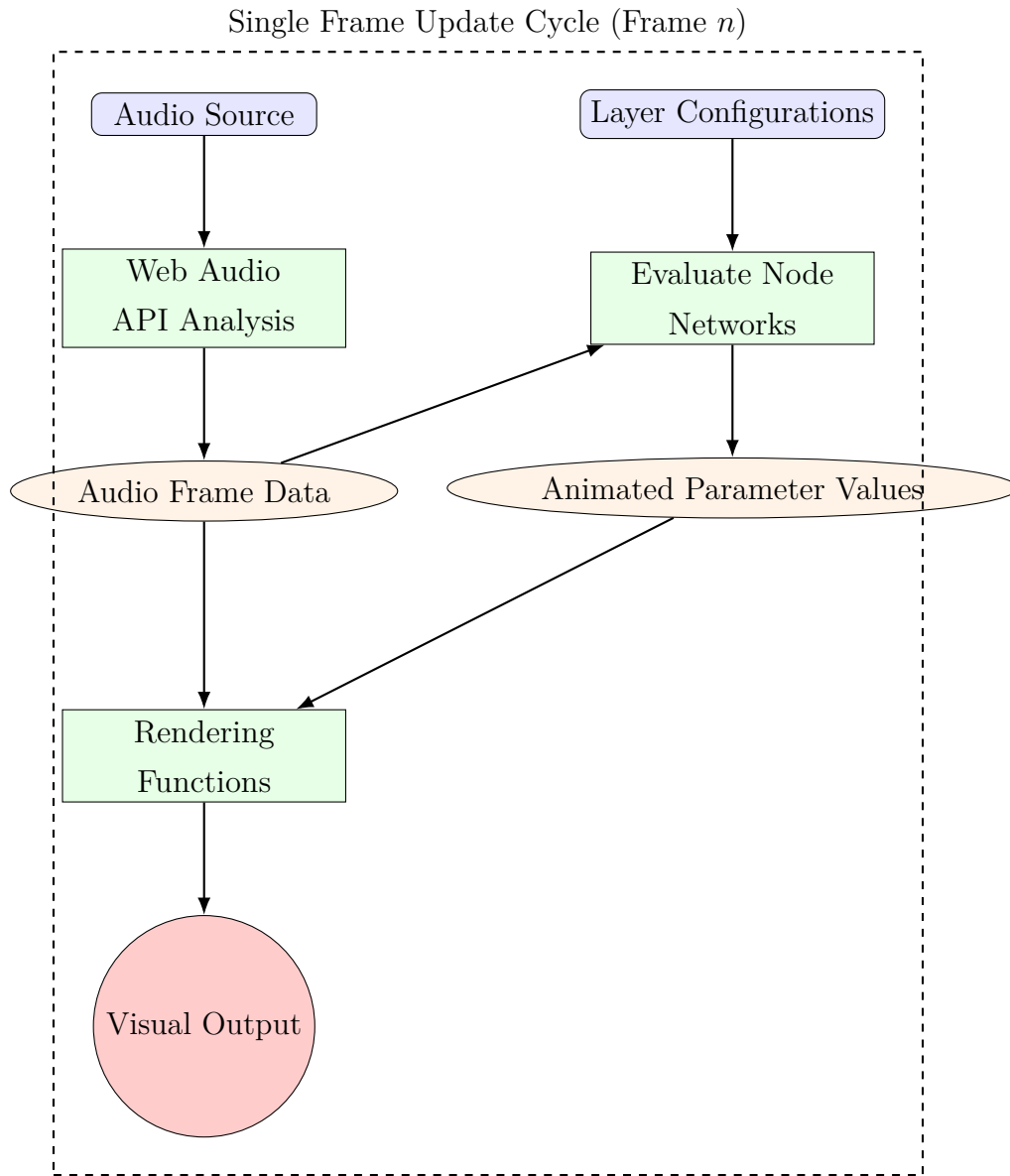


Figure 4.5: The unidirectional data flow within a single frame update cycle.

4.2.3 Component-Centric Rendering Paths

The Visualization Engine must be able to render both 2D and 3D graphics. Rather than enforcing a global rendering mode for an entire project, this choice is made at the component level. Each visual component is a self-

contained template that explicitly declares which rendering context it requires by implementing a specific set of functions.

When a user adds a component as a layer, the engine inspects its definition and provisions the appropriate rendering context for that layer’s dedicated canvas—either an HTML5 Canvas 2D context or a Three.js WebGL renderer. As shown in Listing 2, a 2D component implements a ‘draw’ function, while a 3D component implements ‘init3D’ and ‘draw3D’. This design allows a single composition to mix 2D and 3D layers seamlessly, a key feature for creative flexibility.

```
1  // Example of a component definition declaring its rendering path.
2  const SimpleSpectrum = createComponent({
3    name: 'Simple Spectrum',
4    config: { /* ... parameters ... */ },
5
6    // This component uses the 2D rendering path.
7    draw: ({ canvasCtx, audioData, config }) => {
8      // ... drawing logic using the 2D canvas context ...
9    }
10 });
11
12 const BouncingCube = createComponent({
13   name: 'Bouncing Cube',
14   config: { /* ... parameters ... */ },
15
16   // This component uses the 3D rendering path.
17   init3D: ({ threeCtx }) => {
18     // ... one-time scene setup, e.g., adding a cube ...
19   },
20   draw3D: ({ threeCtx, config, audioData }) => {
21     // ... per-frame logic using the Three.js context ...
22   }
23 });
```

Listing 2: Example definitions for a simple 2D and a 3D component, each declaring its required rendering context via its function signatures.

4.3 The Layer-Based Compositing System

A foundational paradigm in digital graphics software, from video editors to image manipulation programs, is the principle of **compositing**—the process of combining multiple visual elements to create a final image. We adopted this time-tested paradigm for the VizEngine’s core architecture. A final visualization is not a monolithic entity but a **composition** of multiple layers, each contributing a distinct visual element, as depicted in Figure 4.6. This approach allows users to build intricate, evolving scenes by combining and reordering simple, modular components.

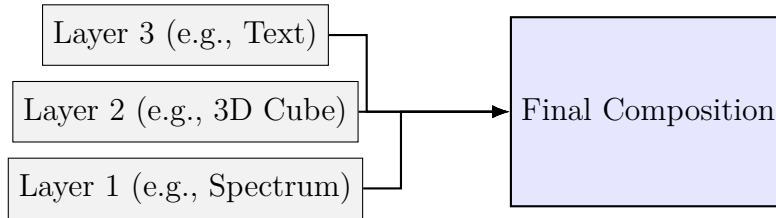


Figure 4.6: A final composition (right) is constructed from a stack of individual, independent layers (left).

4.3.1 The Template-Instance Pattern: From Component to Layer

To manage this system, we employ a **template-instance software design pattern**. A developer first defines a reusable visual blueprint, which we term a **Component**. When a user adds this visual to their scene, the system creates a live, configurable copy of it, which we term a **Layer**.

A Component is a self-contained package, defined as a plain JavaScript object using a ‘createComponent’ factory function. This structure encapsulates all logic and metadata required for a visual element and provides a consistent API for creating new visuals. The key properties of a Component definition are outlined in Table 4.1.

Table 4.1: The core properties of a Component definition object.

Property	Responsibility
<code>name</code>	A unique, human-readable identifier for the component.
<code>description</code>	A brief explanation of the component's visual behavior.
<code>config</code>	A declarative schema defining the user-configurable parameters (e.g. color, size, speed).
<code>defaultNetworks</code>	Optional pre-built animation networks for common parameters.
<code>createState</code>	An optional function that initializes an internal, private state for a new layer instance.
<code>draw</code> / <code>draw3D</code>	The rendering function that draws pixels to a canvas, implementing either a 2D or 3D rendering path.

A concrete example of a simple 3D component definition is shown in Listing 3. When a user selects this Component, a formal instantiation process begins, managed by the system's state stores. This process converts the abstract blueprint into a tangible Layer with a unique identity and a dedicated block of memory for its configuration and state, as shown in the sequence diagram in Figure 4.7.

```
1  const SimpleCube = createComponent({
2    name: 'Simple Cube',
3    description: 'A simple 3D cube that reacts to audio.',
4
5    // Defines the parameters users can edit in the UI.
6    config: v.config({
7      color: v.color({
8        label: 'Cube Color'
9        defaultValue: '#FF6347',
10      }),
11      size: v.number({
12        label: 'Cube Size'
13        defaultValue: 2,
14        min: 0.1,
15        max: 10,
16      })
17    }),
18
19    // This component will use the 3D rendering engine.
20    init3D: ({ threeCtx }) => {
21      // One-time setup: Create a cube mesh and add it to the scene.
22      const geometry = new THREE.BoxGeometry(1, 1, 1);
23      const material =
24        new THREE.MeshStandardMaterial({ color: '#FF6347' });
25      threeCtx.scene.userData.cube =
26        new THREE.Mesh(geometry, material);
27      threeCtx.scene.add(threeCtx.scene.userData.cube);
28    },
29
30    draw3D: ({ threeCtx, config }) => {
31      // Per-frame logic: Update cube based on config values.
32      const cube = threeCtx.scene.userData.cube;
33      if (cube) {
34        cube.scale.setScalar(config.size);
35        cube.material.color.set(config.color);
36      }
37    }
38  });
```

Listing 3: The definition for a basic 3D cube component.

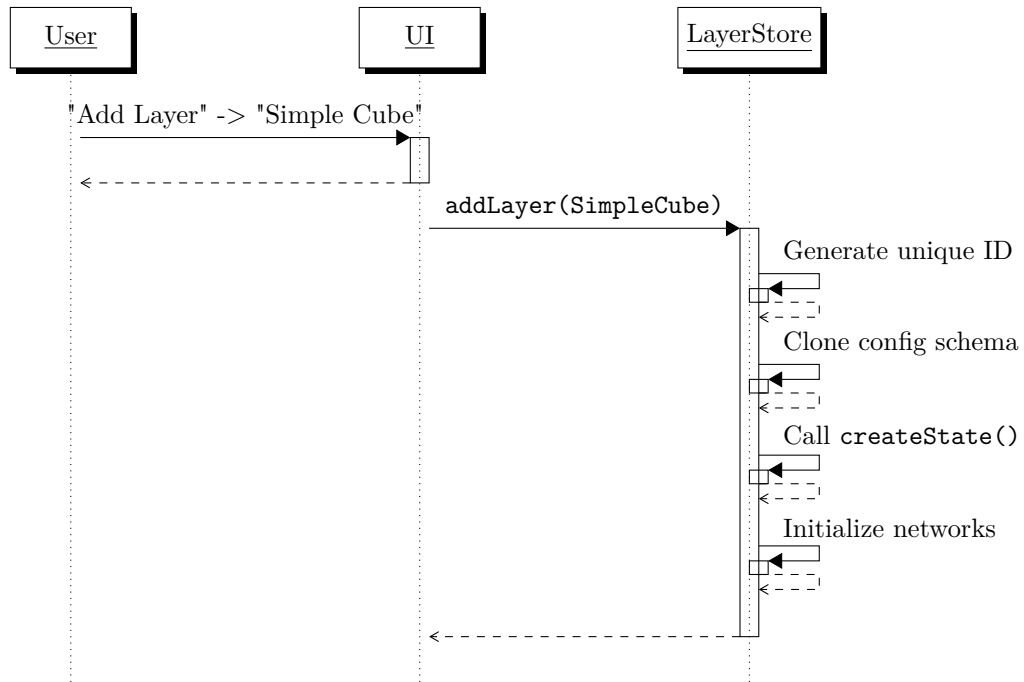


Figure 4.7: The sequence of events when instantiating a Component into a Layer.

4.3.2 The Anatomy of a Layer

Once instantiated, a Layer is represented by a data structure that holds all information pertinent to its existence within the scene. It is critical to distinguish between the three primary data containers on a layer:

- **config:** This object holds the user-facing parameters defined by the component's schema. These values are directly editable by the user through the UI and are the primary targets for animation.
- **state:** This object holds internal data that the component's rendering logic needs to persist across frames but which should not be directly exposed to the user (e.g., the positions of thousands of particles in a particle system).
- **layerSettings:** This object contains metadata that instructs the

compositing engine how to blend this layer with others, such as `opacity`, `blendingMode`, and `isVisible`.

4.3.3 The Multi-Canvas Rendering Pipeline

A key architectural decision was to forgo a traditional, single-canvas rendering target in favor of a **multi-element, DOM-based compositing strategy**. Instead of having all layers draw onto a single, shared canvas, or rendering to off-screen buffers in JavaScript, each layer in the VizEngine is allocated its own dedicated HTML `<canvas>` element.

These canvas elements are positioned within the document's DOM and stacked visually on top of one another using standard CSS positioning properties. Each frame, the engine iterates through the visible layers and provides each component's rendering function with a direct reference to its specific canvas instance to draw upon.

Crucially, the final **compositing** stage is not executed by our application logic in JavaScript. Instead, it is handled natively by the browser's highly optimized rendering engine. We apply CSS properties such as `opacity` and `mix-blend-mode` directly to the individual `<canvas>` elements. The browser then automatically composites these layers according to the standard CSS box model and blending specifications. This architectural approach, illustrated in Figure 4.8, provides two significant advantages:

1. **Isolation and Simplicity.** Each layer operates in a sandboxed environment. A rendering error or performance bottleneck in one layer's canvas context cannot directly affect others. This also simplifies the rendering logic, as components do not need to be aware of a shared global state.
2. **Leveraging Native Browser Performance.** By delegating the compositing task to the browser, we leverage its hardware-accelerated rendering pipeline. This is significantly more performant than implementing a pixel-copying composition loop in JavaScript. It also allows

for seamless integration with other DOM elements (e.g., HTML text or video layers) within the same stacking context.

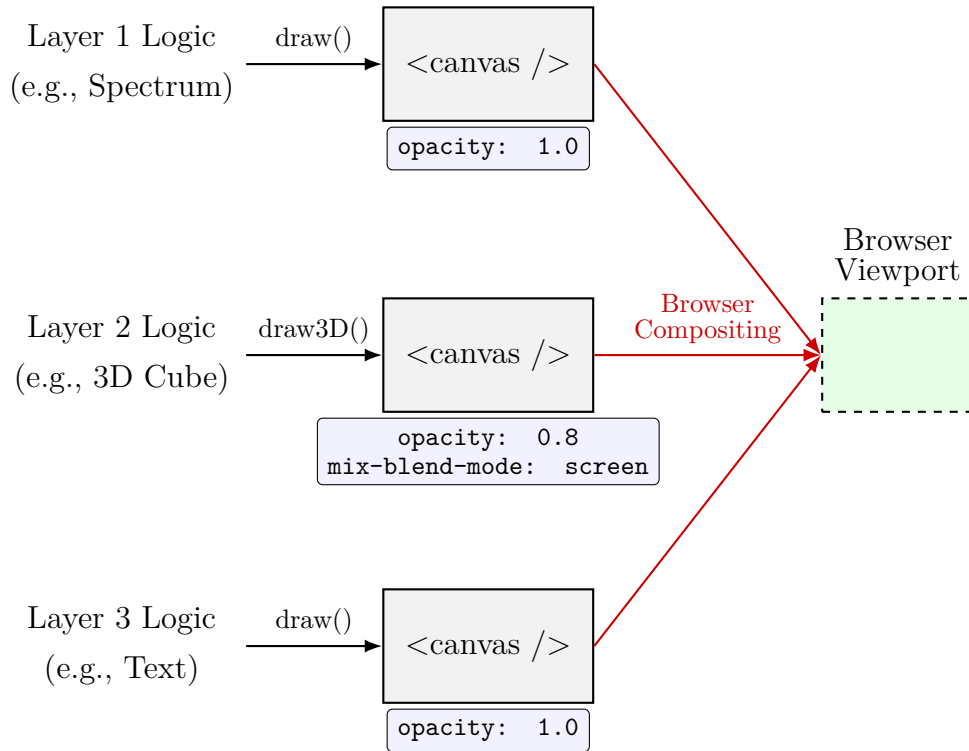


Figure 4.8: The DOM-based compositing pipeline. Each layer’s logic draws to its own canvas element. The browser then composites these stacked elements using their CSS properties.

4.3.4 Layer Lifecycle and Management

The full lifecycle of a layer—from creation and duplication to reordering and eventual destruction—is managed by a dedicated state management store. When a layer is removed, this store ensures that all associated resources, its internal state, and any connected animation networks, are properly de-allocated. This centralized management is crucial for preventing memory leaks in a long-running, dynamic application.

Having established how static visuals are composed, the following section

will introduce the system that animates them: the node-based animation engine.

4.4 The Node-Based Animation Engine

While the layer system provides the static structure for a visual scene, it is the animation engine that provides the mechanism for dynamic behavior. A key architectural decision was to forgo a traditional timeline or keyframe-based animation model in favor of a more expressive approach: a **node-based editor** shown in Figure 4.9. This paradigm, common in professional visual effects and game development software [2, 23], allows users to construct complex behaviors by connecting simple, single-purpose processing blocks, called **nodes**, into a graph. This approach empowers users to design intricate, audio-reactive relationships in an intuitive and exploratory manner.

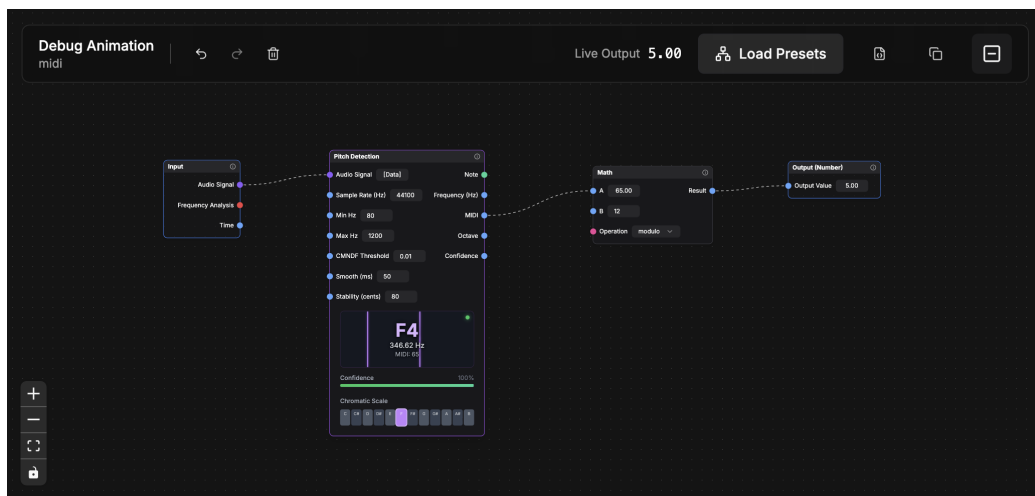


Figure 4.9: A node network within the VizEngine editor. Audio data flows from the Input node (left), is transformed by intermediate nodes, and the final value is sent to the Output node (right), which controls a layer’s parameter.

4.4.1 A Visual Dataflow Paradigm

Every animatable parameter in a layer—be it color, position, or a custom property—can have its own dedicated **node network**. This network is a **directed acyclic graph** that computes the parameter’s value for each frame. The system is a form of visual dataflow programming [29], where data flows through the graph and is transformed at each node, as depicted in Figure 4.10.

A network is composed of three fundamental concepts:

- **Nodes:** The computational units. Each node performs a specific operation, such as a mathematical calculation, a data transformation, or an audio analysis task. It receives data through its input sockets and exposes its results through its output sockets.
- **Edges:** The connections that define the dataflow, linking an output socket of one node to an input socket of another.
- **Terminators:** Every network has special **INPUT** and **OUTPUT** nodes. The **INPUT** node serves as the gateway for global data, such as the full audio signal and timing information. The **OUTPUT** node collects the final computed value and delivers it to the layer parameter being animated.

Node Network for 'size' parameter

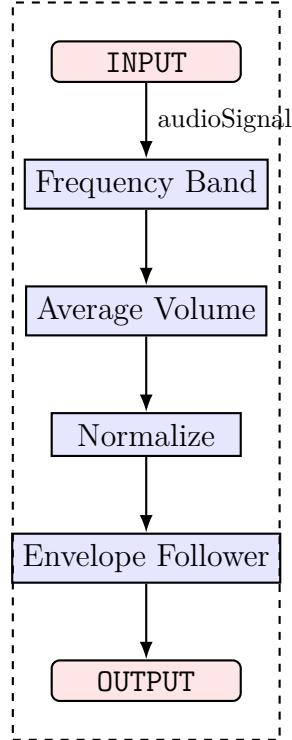


Figure 4.10: A conceptual diagram of a node network that animates a 'size' parameter based on the energy in a specific audio frequency band.

4.4.2 Defining and Discovering Nodes

To ensure the system is both robust and extensible, we established a formal pattern for defining nodes. Each node is a self-contained unit created with a `createNode` factory function. This function takes a definition object specifying the node's metadata, its input and output sockets with their respective data types and default values, and a pure `computeSignal` function containing the node's execution logic.

This declarative, schema-based approach, shown in Listing 4, allows the system to automatically handle UI generation, type validation for connections, and serialization for every node. A primary benefit is that it significantly simplifies the process of extending the engine with new processing capabilities,

as the developer only needs to provide the core logic.

```

1 // Definition for a node that performs a mathematical operation.
2 const MathNode = createNode({
3   label: 'Math',
4   description: 'Performs a math operation on A and B.',
5   // Defines the input sockets and their properties for node UI.
6   inputs: [
7     { id: 'a', label: 'A', type: 'number', defaultValue: 1 },
8     { id: 'b', label: 'B', type: 'number', defaultValue: 1 },
9     { id: 'op', label: 'Operation', type: 'math-op',
10       defaultValue: 'multiply' }
11   ],
12   // Defines the output sockets.
13   outputs: [{ id: 'result', label: 'Result', type: 'number' }],
14   // The core logic: a pure function of the inputs.
15   computeSignal: ({ a, b, op }) => {
16     let result: number;
17     switch (op) {
18       case 'add': result = a + b; break;
19       case 'subtract': result = a - b; break;
20       case 'multiply': result = a * b; break;
21       case 'divide': result = b !== 0 ? a / b : 0; break;
22       default: result = a;
23     }
24     return { result };
25   }
26 });

```

Listing 4: A simplified definition for a versatile Math node, demonstrating the declarative API.

4.4.3 Network Evaluation and Signal Computation

At each frame, the animation engine must compute the output value for every active node network. This is handled by a **demand-driven (lazy), recursive evaluation** algorithm. When a layer requires the value of an animated parameter, the engine requests the value from that parameter's OUTPUT node. The OUTPUT node, in turn, requests the values it needs from the nodes connected to its inputs. This process continues recursively up the

graph until it reaches nodes with no connections or the global `INPUT` node.

To ensure efficiency, this process is heavily optimized with **memoization**. A per-frame cache stores the output of every node as it is computed. If the evaluation algorithm encounters a node whose value is already in the cache, it immediately returns the cached result instead of re-computing it. This has two critical benefits:

1. **Performance:** In complex graphs where one node's output feeds into multiple downstream branches, that node's logic is executed only once per frame.
2. **Circular Dependency Handling:** The caching mechanism inherently prevents infinite loops caused by cycles in the graph. If the algorithm re-encounters a node that is already in the current evaluation stack, it can return a default value from the cache, breaking the recursion.

4.4.4 Advanced Node Capabilities

While many nodes are simple, pure functions, the system also supports more advanced capabilities for sophisticated signal processing.

Stateful Nodes

Some operations, such as smoothing a signal over time, require memory of past frames. We facilitate this by allowing nodes to have an internal, persistent **state** object, initialized via a `createState` function in the node's definition. The `computeSignal` function for a stateful node receives this state, can use its values in its computation, and can return an updated version for the next frame. This enables the implementation of time-based effects like envelope followers and signal integrators, as shown in Listing 5.

```
1 // Snippet from an Envelope Follower node,  
2 // which smooths a signal over time.  
3 computeSignal: ({ value, attack, release }, context, node) => {  
4 // Retrieve persistent state for this specific node instance.  
5 const state = node.data.state as { previousValue: number };  
6 if (state.previousValue === undefined) state.previousValue = 0;  
7  
8 // Use state from the last frame to compute the new value.  
9 const smoothedValue = calculateEnvelope(  
10     value, state.previousValue, attack, release, context.dt  
11 );  
12  
13 // Update the state for the next frame.  
14 state.previousValue = smoothedValue;  
15  
16 return { result: smoothedValue };  
17 }
```

Listing 5: A conceptual example of a stateful node that uses and updates data from the previous frame.

Specialized Audio Nodes

The engine includes a library of specialized nodes for common audio analysis tasks. These nodes abstract the low-level complexity of processing raw FFT data, providing user-friendly building blocks:

- **Frequency Band:** Isolates the energy within a specific frequency range (e.g., 40-120Hz for a kick drum).
- **Spectral Flux:** Detects sharp changes in the audio spectrum, useful for transient detection.
- **Envelope Follower:** Tracks the amplitude of a signal over time, with configurable attack and release.
- **Adaptive Normalize:** Dynamically scales a signal's range based on its recent minimum and maximum values.

By providing these powerful primitives as simple nodes, the system makes advanced audio-reactive techniques accessible to non-programmers. This visual programming paradigm is the engine that drives the dynamic nature of the visualizations, transforming static layers into elements that respond directly to the audio.

4.5 The Audio Processing Pipeline

The foundation of any audio-reactive system is its ability to process audio signals. The VizEngine’s audio pipeline is designed to be a precise and high-performance subsystem capable of handling both pre-recorded audio files and live streams. Its primary responsibility is to capture an audio signal and transform it into a structured, frame-synchronized stream of feature data that can drive the animation engine, as detailed in Section 4.4. Figure 4.11 shows the audio control panel with displayed audio waveform and controls for playback.

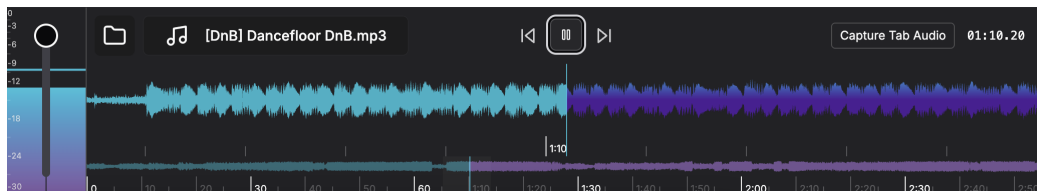


Figure 4.11: The audio control panel, which uses the WaveSurfer.js library to provide users with a visual representation of the audio waveform and controls for playback.

4.5.1 The Dual-Path Audio Graph

A key architectural decision was to **decouple audio analysis from audio playback**. Routing a single audio stream through analysis nodes (such as an ‘AnalyserNode’ for FFT computation) before it reaches the speakers can introduce unwanted latency or audible artifacts. Furthermore, the ideal

settings for signal analysis (e.g., zero temporal smoothing for sharp transient detection) are often detrimental to a pleasant listening experience.

To resolve this, we implemented a dual-path architecture using the Web Audio API, as shown in Figure 4.12. When an audio source is connected to the audio context, its output is split. One path is routed directly to the audio destination for high-fidelity playback. A parallel, independent path is routed to a dedicated ‘AnalyserNode’ for real-time data extraction. This analysis node is intentionally disconnected from the final output, which prevents any "double audio" or phasing issues and ensures that the analysis process is non-destructive to the playback signal.

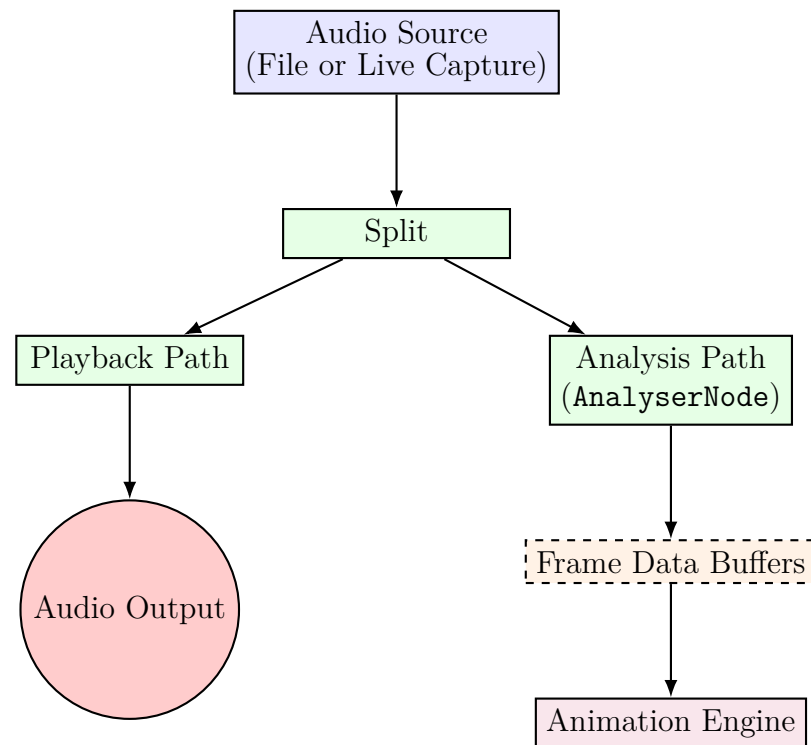


Figure 4.12: The dual-path audio graph. The signal is split to serve playback and analysis independently, ensuring both high-fidelity output and precise, non-destructive analysis.

4.5.2 Audio Source Management

The system is designed to be source-agnostic, capable of switching between two primary types of audio input.

File-Based Audio The primary mode of operation involves user-provided audio files. For this, we integrated the **WaveSurfer.js** library to handle file loading, decoding, and the rendering of a visual waveform. This provides users with a familiar interface for navigating the audio timeline. Playback is synchronized with the main composition’s clock to ensure visual events and audio cues are precisely aligned.

Live Audio Capture For generative or live performance scenarios, the engine can capture audio from other browser tabs or applications using the browser’s native `getDisplayMedia` API. A critical implementation detail, shown in Listing 6, is to explicitly request that the browser disables its default audio processing features. Echo cancellation and noise suppression, while useful for communication applications, can distort the signal and interfere with the accuracy of the analysis pipeline.

```
1 // Requesting a clean audio stream for analysis.
2 const stream = await navigator.mediaDevices.getDisplayMedia({
3   audio: {
4     // These constraints are disabled
5     // to get the rawest signal possible.
6     echoCancellation: false,
7     noiseSuppression: false,
8     autoGainControl: false,
9   },
10  // We request video but do not use it,
11  // as it is required by the API.
12  video: true
13 });
14
15 // Create a Web Audio API source from the live stream.
16 const source = audioContext.createMediaStreamSource(stream);
17
18 // Connect the live source to the analysis graph.
19 source.connect(audioAnalyzer);
```

Listing 6: Capturing a high-quality, unprocessed audio stream from another application for live visualization.

4.5.3 The Real-Time Analysis Pipeline

Once an audio source is connected to the ‘AnalyserNode’, the pipeline begins its per-frame analysis. The ‘AnalyserNode’ performs a Fast Fourier Transform (FFT) to provide frequency-domain data and can also provide raw time-domain (waveform) data. We configured the analyzer with specific parameters optimized for visualization:

- **FFT Size:** An `fftSize` of **2048** was chosen as it provides a suitable balance between frequency resolution (yielding 1024 frequency bins) and temporal responsiveness. Assuming the common sample rate of 44.1 kHz (which is used for this thesis’s offline analysis, see Section 4.9), this `fftSize` provides a frequency resolution of approximately **21.53 Hz per bin** ($\frac{44100 \text{ Hz}}{2048}$).

- **Smoothing:** The `smoothingTimeConstant` is set to `0`. This disables the default temporal averaging between FFT buffers. While smoothing can produce a more aesthetically pleasing spectrum, its absence is critical for accurate, instantaneous onset detection.
- **Decibel Range:** The range is set from **-90dB to -10dB**. This was determined empirically to better capture the dynamic range of typical musical recordings.

At the beginning of each frame update cycle, the system calls ‘`getBytesFrequencyData()`’ and ‘`getBytesTimeDomainData()`’ on the analyzer. To optimize performance and minimize pressure on the garbage collector, the `Uint8Array` buffers required by these methods are created only once and are reused on every subsequent frame, as shown in Listing 7.


```
1 // Pre-allocated buffers are held in a React ref
2 // to persist across renders.
3 const frequencyDataRef = useRef<Uint8Array>(
4   new Uint8Array(analyzer.frequencyBinCount)
5 );
6 const timeDomainDataRef = useRef<Uint8Array>(
7   new Uint8Array(analyzer.fftSize)
8 );
9
10 // This function is called once per frame.
11 const captureFrameData = () => {
12   if (isPlaying) {
13     // The analyzer populates the buffers with the latest data.
14     // No new memory is allocated in this critical path.
15     analyzer.getBytesFrequencyData(frequencyDataRef.current);
16     analyzer.getBytesTimeDomainData(timeDomainDataRef.current);
17   }
18   return {
19     frequencyData: frequencyDataRef.current,
20     timeDomainData: timeDomainDataRef.current,
21   };
22 };
```

Listing 7: Efficient per-frame capture of audio data using pre-allocated and reused buffers to minimize memory pressure.

This packet of frame-synchronized audio data is then made available to the system, serving as the primary input for the node-based animation engine. This closes the loop from audio input to visual output, enabling the system’s core audio-reactive functionality.

4.6 The Declarative Configuration and UI System

A key challenge in creating an extensible creative tool is to provide intuitive controls for user-defined components without tightly coupling the component’s logic to a specific UI implementation. To solve this, we developed

a **declarative configuration system**. Instead of manually constructing UI forms for each component, a developer simply defines a **configuration schema**. This schema serves as a single source of truth that the system uses to dynamically generate interactive controls, manage parameter values, and expose hooks for the animation engine.

4.6.1 The Schema as a Single Source of Truth

When developing a new component, the author defines its user-controllable parameters using a fluent, type-safe API. This API facilitates the definition of various parameter types, such as numbers, colors, and booleans, each with specific constraints like minimum/maximum values and default settings.

This schema-first approach is central to the system’s extensibility. The component author focuses on *what* parameters are needed and their logical constraints, not *how* they should be rendered in the UI. An example of a more complex schema is shown in Listing 8. This schema defines nested parameter groups and includes conditional visibility logic, where one parameter’s visibility depends on the value of another.

```
1  const particleConfig = v.config({
2    appearance: v.group({ label: 'Appearance' }, {
3      startColor: v.color({
4        label: 'Start Color',
5        defaultValue: '#ff00ff'
6      }),
7      endColor: v.color({
8        label: 'End Color',
9        defaultValue: '#00ffff'
10     }),
11     particleSize: v.number({
12       label: 'Particle Size',
13       description: 'Size of individual particles',
14       defaultValue: 0.2,
15       min: 0.1,
16       max: 5,
17       step: 0.1,
18     }),
19     blending: v.select({
20       label: 'Blending Mode',
21       description: 'How particles blend together',
22       defaultValue: 'additive',
23       options: ['additive', 'normal', 'multiply'],
24     }),
25   }),
26   physics: v.group({ label: 'Physics' }, {
27     emissionRate: v.number({
28       label: 'Rate',
29       defaultValue: 100,
30       min: 0,
31       max: 1000
32     }),
33     lifetime: v.number({
34       label: 'Lifetime (s)',
35       defaultValue: 2,
36       min: 0.1,
37       max: 10
38     }),
39     useGravity: v.toggle({
40       label: 'Use Gravity',
41       defaultValue: true
42     }),
43     // This parameter's visibility is conditional.
44     gravityStrength: v.number({
45       label: 'Gravity Strength',
46       defaultValue: 9.8,
47       visibleIf:
48         (allValues) => allValues.physics.useGravity === true
49     })
50   })
51 });
```

Listing 8: A declarative schema defining a component's parameters, including

4.6.2 Dynamic UI Generation

The system contains a **form generator** that introspects a component's configuration schema at runtime and builds a corresponding user interface with React. It maps each parameter type in the schema to a specific, pre-built UI control:

- `v.number()` maps to a slider and a precise number input.
- `v.color()` maps to a color picker.
- `v.toggle()` maps to a switch.
- `v.group()` maps to a collapsible accordion section.

This automatic generation ensures a consistent user experience across all components and dramatically reduces the boilerplate code required to create new visualizations. The form generator handles its own state, validation, and updates, freeing the component developer to focus solely on the rendering logic. Figure 4.13 shows the UI generated from the `particleConfig` schema.

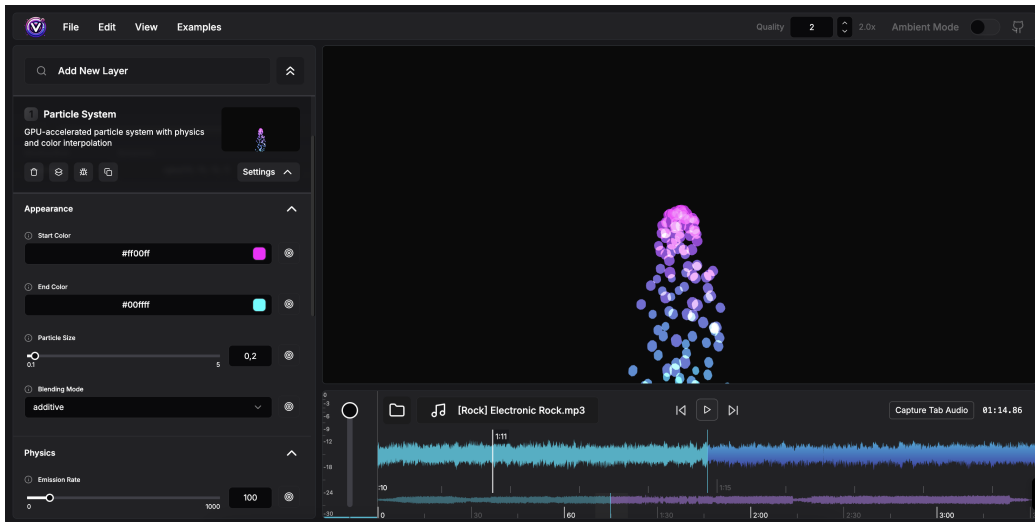


Figure 4.13: The UI generated from the `particleConfig` schema. The "Gravity Strength" slider is only visible because "Use Gravity" is enabled, as per the `visibleIf` condition.

4.6.3 Bridging Static Values and Live Animation

A parameter's value can originate from one of two sources: a **static value** set manually by the user in the form, or a **live value** computed each frame by a node network. The configuration system is responsible for managing this duality.

Each parameter control in the generated UI is rendered with an "animation" toggle. When this toggle is disabled, the parameter's value is read directly from the form's state store. When a user enables the toggle, a new node network is created and associated with that specific parameter, and the UI control becomes disabled, instead displaying the live animated value.

The component's rendering logic remains agnostic to the source of its parameter values. It simply requests a value from the configuration system, which acts as a gatekeeper. As shown in Figure 4.14, the system checks if animation is enabled for the requested parameter. If it is, it executes the corresponding node network and returns the result; otherwise, it returns the static value from the form. This architecture cleanly separates the concerns of static configuration and dynamic animation, providing a flexible workflow for the user.

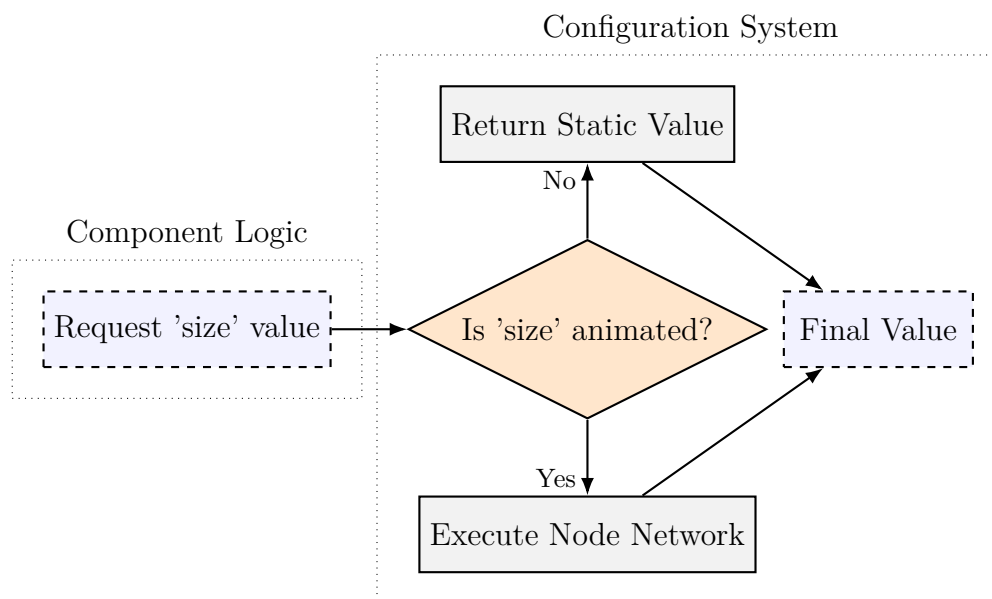


Figure 4.14: The value resolution process for the 'size' parameter. The configuration system transparently provides either a static or a live animated value to the component.

4.7 The State Management Architecture

A real-time graphics application is fundamentally a state machine. Every user interaction—adding a layer, adjusting a parameter, connecting a node—mutates this state, which in turn dictates the visual output on the next frame. Managing this complexity is a critical architectural challenge. For the VizEngine prototype, we selected the **Zustand** library for its minimalism and performance characteristics. We built our architecture around a **multi-store pattern**, where the application's global state is partitioned into several independent, domain-specific stores.

This approach contrasts with maintaining a single, monolithic state tree. By splitting the state, we achieve a clear separation of concerns and a significant performance benefit: a React component can subscribe to updates from only the specific store it requires. For example, a UI component that

adjusts layer visibility will not re-render when the audio source changes, as those pieces of state reside in different stores.

4.7.1 A List of the State Stores

The system's state is distributed across the core stores detailed in Table 4.2, each with a precisely defined responsibility. This distributed architecture can be visualized as a set of cooperating services, where UI components subscribe only to the stores relevant to their function, as shown in Figure 4.15.

Table 4.2: Core state stores and their respective domains and responsibilities.

Store	Domain and Responsibilities
LayerStore	Domain: Visualization Manages the array of <code>LayerData</code> objects; handles creation, deletion, duplication, and reordering of layers.
NodeNetworkStore	Domain: Animation Holds the graph structure (nodes and edges) for every animated parameter; triggers network evaluation.
AudioStore	Domain: Audio I/O Manages the Web Audio API context, the current audio source (file or live), and the <code>AnalyserNode</code> .
EditorStore	Domain: Application & UI Contains global UI state, such as playback status (play/pause), quality settings, and references to the player.
ValueStores	Domain: Parameter Data A collection of smaller stores holding the static values for layer configurations and node inputs.

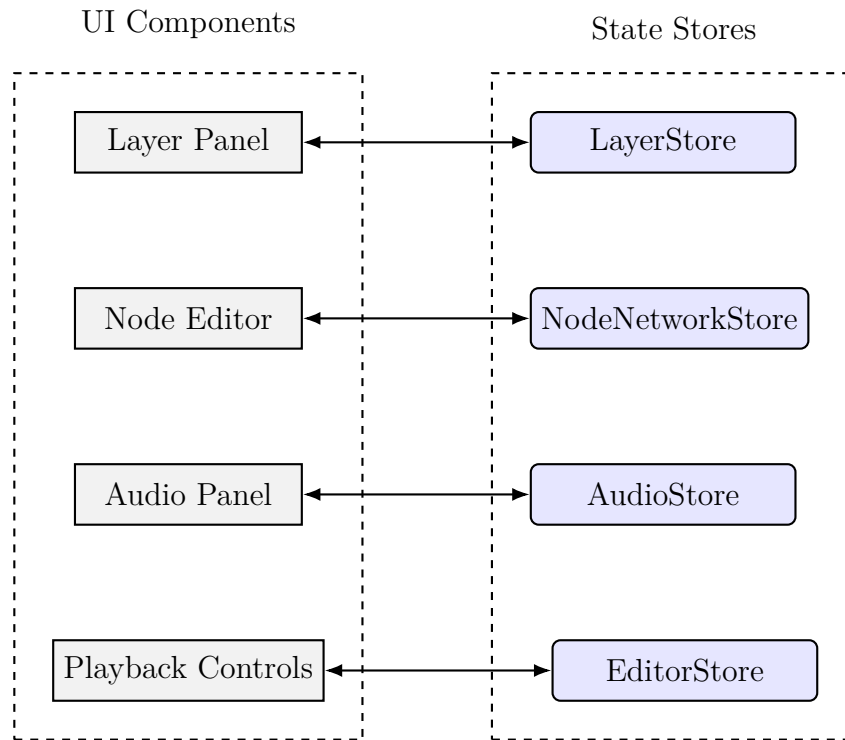


Figure 4.15: High-level view of UI components subscribing to their relevant state stores.

4.7.2 Cross-Store Communication and Data Flow

While the stores are independent, they must communicate to execute complex operations. This is achieved not through a central dispatcher or event bus, but through direct, imperative function calls between stores. A prime example is the creation of a new layer, an action which originates in the 'LayerStore' but has cascading effects:

1. The 'LayerStore's 'addLayer' function is called.
2. It creates the new 'LayerData' object and adds it to its internal list.
3. It then calls a function in the appropriate **ValueStore** to initialize the new layer's static parameter values based on the component's defaults.

4. Finally, it inspects the component for any ‘defaultNetworks’ and, if present, calls the **NodeNetworkStore**’s ‘setNetwork’ function to instantiate those default animation networks.

This chain of responsibility ensures that when a layer is created, it is fully initialized across all relevant state domains. A similar, inverse process occurs upon layer deletion to ensure proper cleanup of all associated data.

4.7.3 Performance and Subscription Optimization

The primary performance benefit of the multi-store architecture lies in enabling **selective subscriptions**. A React component that only needs to know if the audio is playing can subscribe to just that single boolean value from the ‘EditorStore’, ignoring all other state changes in the application.

Zustand facilitates this through the use of selector functions. As demonstrated in Listing 9, a component can provide a function to the ‘useStore’ hook, which extracts only the necessary slice of state. The library ensures that the component will only re-render if the return value of this selector function changes. We enforce this pattern throughout the application to minimize re-renders and maintain a responsive user interface, which is critical while the editor to render at 60 FPS.

```

1  // A React component subscribing to a single,
2  // specific piece of state.
3  const PlayPauseButton = () => {
4      // This component will only re-render
5      // when the `isPlaying` boolean changes.
6      const isPlaying = useEditorStore((state) => state.isPlaying);
7      const setIsPlaying = useEditorStore((state) => state.setIsPlaying);
8
9      return (
10         <button onClick={() => setIsPlaying(!isPlaying)}>
11             {isPlaying ? 'Pause' : 'Play'}
12         </button>
13     );
14 };
15
16 // An anti-pattern: subscribing to the entire store object.
17 const InefficientButton = () => {
18     // This would cause the component to
19     // re-render on ANY change in the store.
20     const editorStore = useEditorStore(); // This should be avoided.
21     // ...
22 };

```

Listing 9: Demonstrating efficient, selective subscription with Zustand selectors to minimize React re-renders.

4.7.4 State Persistence and Hydration

To provide a seamless user experience, the entire state of a project is automatically saved to the browser’s **IndexedDB** storage. We use Zustand’s ‘persist’ middleware to handle this. However, not all application state is serializable or desirable to persist (e.g., live references to ‘<canvas>’ elements or the ‘AudioContext’). We implement custom **partialization** and **merge** functions to manage this.

- **Partialization:** Before writing to the database, a ‘partialize’ function is run on the state. It creates a serializable subset of the state, stripping out transient data and transforming complex objects into a storable

format (e.g., storing a component reference by its unique name string instead of the object itself).

- **Merging:** When the application loads, the persisted state is read from IndexedDB. A custom ‘merge’ function then intelligently combines this persisted data with the initial, default state of the application, re-establishing references and reconstructing the full, live application state.

This robust persistence layer ensures that a user can close their browser and return to their work exactly where they left off, creating a more professional and reliable user workflow.

4.8 Project Persistence and Data Management

A creative tool is only as useful as its ability to reliably save a user’s work. For the VizEngine prototype, we designed a **two-tiered persistence strategy** to cater to two distinct user needs: the need for work to be saved automatically during a session, and the need to export and share a project as a self-contained file.

4.8.1 Two-Tier Persistence Strategy

Tier 1: Automatic Session Persistence (IndexedDB) To prevent data loss and provide a fluid user experience, the application’s state is automatically and continuously saved to the browser’s **IndexedDB**. This functions as a robust "autosave" feature. We implemented a **throttled writer** that batches state changes and commits them to the database every 100 milliseconds. This approach prevents excessive disk I/O during rapid user interactions (such as adjusting a slider) while ensuring that recent changes are reliably saved. This local, browser-based storage is the foundation of our session management.

Tier 2: Manual Project Portability (File Export/Import) For sharing, versioning, and backup, users can manually export their entire project to a single JSON file. This process gathers the serializable state from all relevant stores, packages it with a version number, and bundles it for download. A user can then import this file into any instance of the VizEngine to replicate the saved project state. This ensures that projects are portable and not locked to a single browser or machine. The relationship between these two tiers is illustrated in Figure 4.16.

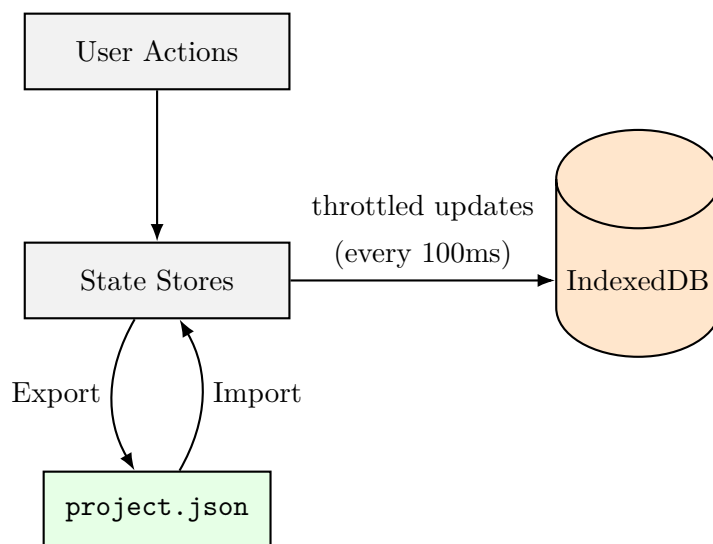


Figure 4.16: The two-tier persistence model, showing both automatic background saving to IndexedDB and manual file-based import/export operations.

4.8.2 The Serialization Challenge

A significant challenge in saving application state is that certain objects cannot be serialized into standard formats like JSON. Live objects such as the ‘AudioContext’, WebGL renderers, or references to HTML elements are non-serializable and must be reconstructed when a project is loaded.

Our solution is to implement custom serialization (**partialize**) and deserialization (**merge**) logic within our state stores, as discussed in Section 4.7.4.

- When saving, the `partialize` function creates a "dehydrated" version of the state. It strips out live, non-serializable references and converts complex objects into a storable format. For example, a reference to a full Component object is replaced with a simple string containing that component's unique name.
- When loading, the `merge` function "rehydrates" this data. It reads the dehydrated state and, using the component name string, looks up the full Component definition from the live component registry to restore the original reference.

This process, illustrated in Listing 10, ensures that the loaded project is correctly linked to the current, running version of the application code.

```
1 // A simplified example of a live layer object in memory.
2 const liveLayer = {
3   id: 'cube-1',
4   // Contains the full component object, including functions.
5   comp: SimpleCubeComponent,
6   // Contains a non-serializable reference to a canvas element.
7   mirrorCanvas: HTMLCanvasElement,
8   // ... other data
9 };
10
11 // The partialize function transforms it for serialization.
12 function partialize(layer) {
13   return {
14     id: layer.id,
15     // The component is referenced only by its unique name string.
16     compName: layer.comp.name,
17     // The non-serializable canvas reference is stripped out.
18     // ... other serializable data
19   };
20 }
```

Listing 10: Transforming a live Layer object into a serializable format for persistence by removing transient references and reducing objects to identifiers.

4.8.3 Future Work: Cloud Persistence and Collaboration

The current architecture, with its well-defined state stores and serialization logic, provides a strong foundation for moving beyond local persistence to a fully online, collaborative platform. The serialized JSON project file serves as a suitable Data Transfer Object (DTO) for a backend API.

A future version of the engine could integrate with a real-time database provider (such as Firebase [30] or Supabase [31]). This would enable several powerful features:

- **Cloud Storage:** Users could save projects to an account and access them from any device.
- **Real-time Collaboration:** Multiple users could work on the same project simultaneously, with state changes from one user being broadcast to all other collaborators, utilizing platforms like Convex [32].
- **Project Sharing:** Users could share a URL that links directly to a project, enabling a public gallery or community showcase.

This potential evolution would transform the VizEngine from a standalone, client-side tool into a collaborative, web-native creative suite.

4.9 The Export Pipeline

While real-time playback is the primary mode of interaction, a creative tool requires a robust method for exporting high-quality, shareable video or image files. The architectural challenge is that the visual output of VizEngine is not a simple video stream but the result of two complex, real-time processes:

1. **Stateful Animation:** Visuals, especially physics-based or generative ones, are time-dependent. The state of a frame n depends on the state of frame $n - 1$.

2. **DOM-Based Compositing:** The final image is a composite of multiple `<canvas>` elements, blended by the browser's CSS engine, as described in Section 4.3.

A simple screen recording would be locked to the user's display resolution and real-time performance, while a non-stateful, frame-based rendering engine (like Remotion's) would not support our stateful, generative components. To solve this, we designed a custom, multi-phase, client-side export pipeline that programmatically re-renders the visualization frame-by-frame, performs manual composition, and encodes the result using WebAssembly.

4.9.1 A Multi-Phase Export Architecture

The export system is orchestrated as an asynchronous pipeline that executes in three distinct phases, as illustrated in Figure 4.17. This process runs entirely within the user's browser, requiring no server-side resources.

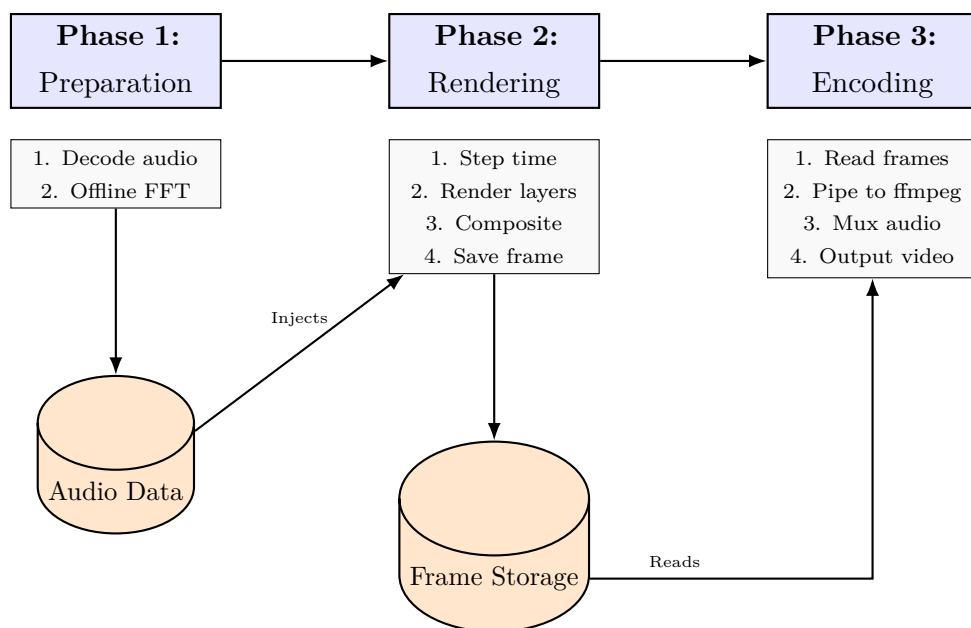


Figure 4.17: The three-phase architecture of the client-side export pipeline.

4.9.2 Phase 1: Preparation and Offline Audio Analysis

During live playback, audio features are extracted in real-time using the Web Audio API's `AnalyserNode`. However, this node only functions on a live `AudioContext` and cannot provide deterministic, frame-accurate data for an offline render.

To solve this, the preparation phase performs a one-time analysis of the entire audio source. First, an `OfflineAudioContext` is used to decode the compressed audio file (e.g., MP3) into raw PCM sample data. Then, a custom JavaScript-based **Fast Fourier Transform (FFT)** implementation, based on the foundational Cooley-Tukey algorithm [9], analyzes the precise audio chunk for each video frame, as shown in Table 4.3.

The system calculates the exact audio samples corresponding to each frame using the formula in Equation 4.3:

$$\text{samplesPerFrame} = \frac{\text{sampleRate}}{\text{fps}} \quad (4.3)$$

For instance, at 60 FPS with a 44.1 kHz sample rate, each frame analyzes 735 audio samples (approximately 16.7 ms of audio). The resulting frequency and time-domain data for every single frame is cached in memory. This ensures that when the renderer asks for audio data for frame n , it receives the exact, deterministic data corresponding to that frame's time window.

Table 4.3: Deterministic audio-to-frame mapping in the offline extractor (for 60 FPS at 44.1 kHz sample rate).

Video Frame	Sample Range	Time Range	Audio Chunk
Frame 0	0 - 735	0.000s - 0.0167s	735 samples
Frame 1	735 - 1470	0.0167s - 0.0333s	735 samples
Frame 2	1470 - 2205	0.0333s - 0.0500s	735 samples
...	...	...	...

4.9.3 Phase 2: Frame-by-Frame Rendering and Compositing

This phase is the core of the export pipeline. Instead of relying on the browser's real-time `requestAnimationFrame` loop, an **Export Orchestrator** takes control of the system clock. It iterates from frame 0 to the final frame, and for each frame, it performs the following steps:

1. **Manual Time-Stepping:** It calculates the exact `currentTime` and `deltaTime` (e.g., $\frac{1}{60}s$).
2. **Audio Injection:** It injects the pre-calculated audio data for the current frame into the `AudioStore`. The `useAudioFrameData` hook is designed to automatically consume this offline data when it detects the system is in "export mode."
3. **Stateful Render:** It programmatically calls the `renderAllLayers(t, dt)` function, triggering all component rendering logic and stateful updates.
4. **GPU Synchronization:** For 3D layers using WebGL, it explicitly calls `gl.finish()`. This command blocks the main thread until the GPU has completed all rendering operations, preventing the capture of incomplete frames.
5. **Manual Compositing:** Since the browser's CSS compositing is not available, a custom "fast capture" function manually replicates it. It iterates through all layer `<canvas>` elements, reads their CSS `opacity` and `mix-blend-mode` properties, and draws them sequentially onto a single, off-screen composite canvas, using the Canvas 2D API's `globalCompositeOperation` to emulate the blend modes. This manual compositing is a practical implementation of the alpha-blending and compositing algebra formally defined by Porter and Duff [33].

6. **Frame Storage:** The final composited frame is encoded as a JPEG and saved to **IndexedDB**. Using IndexedDB (Table 4.4) is a critical memory management technique that moves frame data from RAM to disk, enabling the export of long, high-resolution videos without crashing the browser.

Table 4.4: Estimated IndexedDB storage requirements for a 60-second, 60 FPS video export (3600 frames) using JPEG compression.

Resolution	Est. Size per Frame	Est. Total Storage
720p (1280x720)	$\approx$ 50 KB	$\approx$ 180 MB
1080p (1920x1080)	$\approx$ 100 KB	$\approx$ 360 MB
4K (3840x2160)	$\approx$ 400 KB	$\approx$ 1.4 GB

4.9.4 Phase 3: Client-Side Encoding with WebAssembly

Once all frames are rendered and stored in IndexedDB, the final phase begins. The system loads **FFmpeg.wasm**, a WebAssembly port of the complete FFmpeg video processing suite. The ability to compile complex, native C/C++ applications like FFmpeg into a high-performance, browser-compatible format is a direct result of the WebAssembly architecture [34]. The orchestrator reads the frames back from IndexedDB and writes them, along with the original audio file, into FFmpeg’s virtual in-memory filesystem.

An FFmpeg command is then executed—entirely within the browser—to process these inputs. This command multiplexes the image sequence and the audio track, applies the correct frame rate, and encodes the final video into standard formats like H.264 (MP4).

4.9.5 Architectural Justification

The decision to build this complex, custom export pipeline was a deliberate architectural pivot from the initial plan, which was to use an existing library like **Remotion**. Remotion’s architecture was initially highly appealing, as it is optimized for **massively parallel rendering** and offers a cloud-based service, **Remotion Lambda**, which leverages AWS to render videos at high speeds. This server-side, parallel approach was seen as the ideal path to fast, scalable, and high-quality exports.

However, a deeper analysis revealed a fundamental paradigm conflict. Remotion’s parallelism requires that every frame be **deterministic and stateless**, meaning its output can be calculated based *only* on a frame number.

This paradigm directly conflicts with a core design goal of VizEngine: to support complex, **stateful, and physics-based animations** where a frame’s content explicitly depends on the state of the previous frame. Adopting a Remotion-style export model would have required a fundamental rewrite of the component architecture, forcing developers to abandon stateful logic and dramatically limiting the system’s creative potential.

The custom pipeline, while slower to export on the client side, is architecturally more appropriate for this project because it runs *on top* of the existing, stateful system. It honors the developer’s creative intent and allows for the export of complex, generative, and non-deterministic visuals without compromise. This aligns with the project’s long-term vision of supporting intricate, emergent behavior, such as live-reactive installations or even future collaborative, multiplayer virtual environments.

4.10 The Editor UI and User Experience

The efficacy of the underlying engine is contingent upon an intuitive and efficient control surface. VizEngine’s user interface is therefore designed to support a fluid, exploratory workflow, enabling users to move seamlessly

between composing scenes, configuring layers, and designing complex animations. The entire editor is implemented as a single-page application (SPA) with a flexible, multi-panel layout, as shown in Figure 4.18.

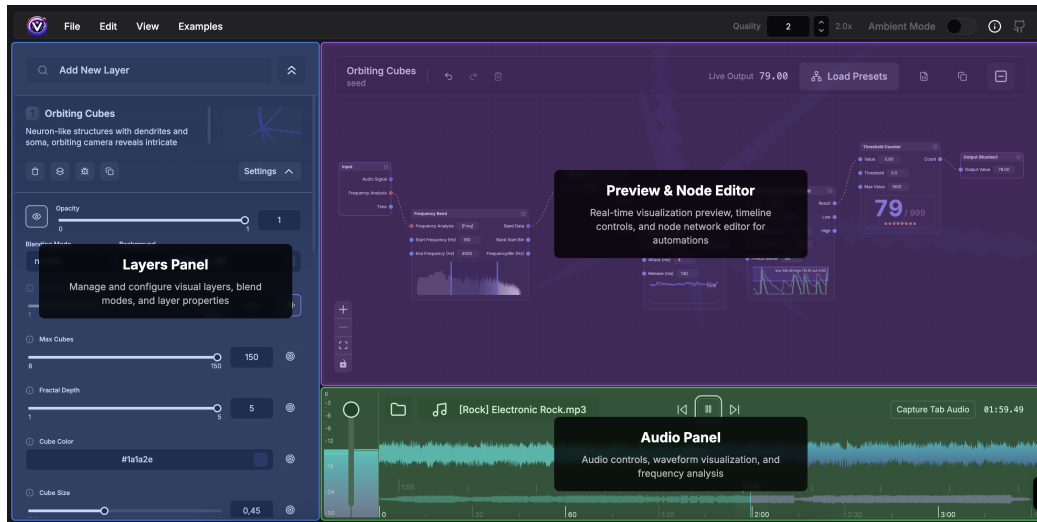


Figure 4.18: The main editor layout, divided into a layer management panel (left) and a primary workspace with the canvas, animation editor, and audio controls (right).

4.10.1 A Multi-Panel Workspace

The foundation of the editor is a resizable panel system that allows users to customize their workspace. By default, the layout is organized into three primary zones:

- **The Left Panel** is dedicated to scene composition and layer management.
- **The Top-Right Panel** contains the main visual output canvas.
- **The Bottom-Right Panel** houses contextual tools, primarily the audio timeline and the node-based animation editor.

This division is a deliberate design choice intended to minimize context switching by keeping the most relevant information and controls simultaneously visible.

4.10.2 The Layer Management Panel

The process of scene composition is initiated and managed within the left panel. This is where users discover available visual Components and manage the instantiated Layers in their project. The top of the panel contains a search interface for adding new components to the scene. Below this is the **layer stack**, a vertical, sortable list of all layers in the project, shown in Figure 4.19. The top-to-bottom order of this list corresponds directly to the rendering order of the layers.

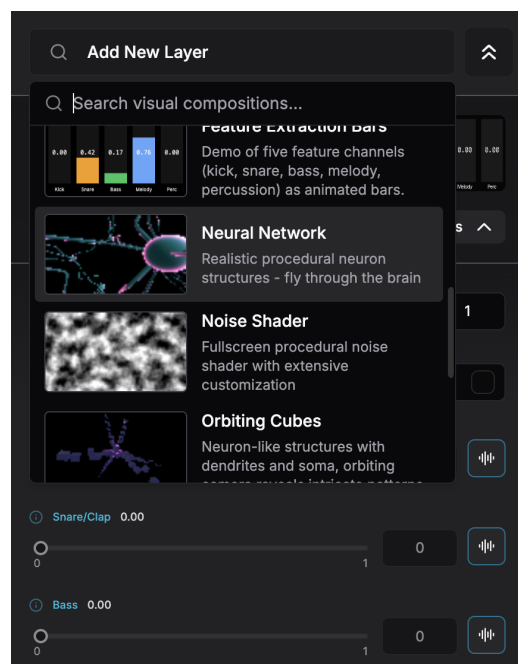


Figure 4.19: The layer management panel, showing the "Add New Layer" interface and a stack of configurable layer cards.

Each layer is represented by a collapsible card. In its collapsed state, the card provides essential controls: a drag handle for reordering, a visibility toggle,

and a live thumbnail preview of that layer's isolated content. Expanding the card, as shown in Figure 4.20, reveals the full set of controls. This includes general settings like **opacity** and **blend mode**, as well as the dynamically generated form for the component's specific parameters, as detailed in Section 4.6.

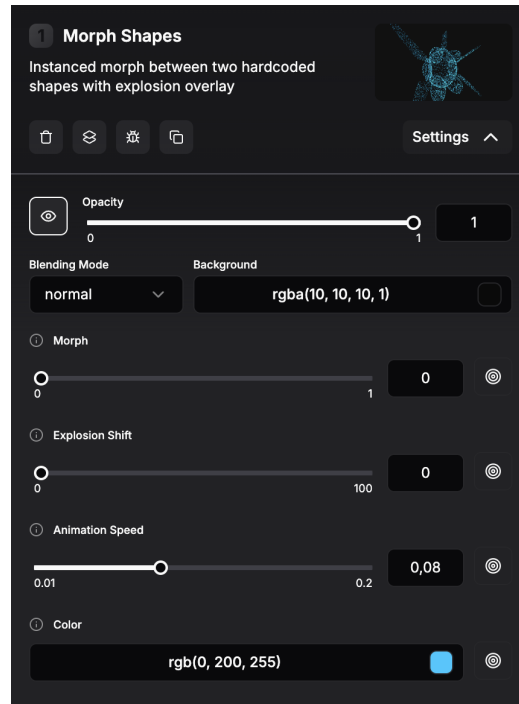


Figure 4.20: An expanded layer card, revealing the dynamically generated form with sliders and color pickers for the component's parameters.

4.10.3 The Primary Workspace

The right side of the editor contains the main canvas and the animation tools. The largest panel is a **custom-built player component** that displays the final, composited visual output. This component was developed in-house to replace a third-party player, gaining two significant advantages: **performance**, by minimizing component re-renders during the 60 FPS animation loop; and **flexibility**, by allowing for a fully custom interface adapted for the application's specific audio input modes.

When a user elects to animate a parameter, the **Animation Builder** appears as a semi-transparent overlay on top of this player, as seen in Figure 4.21. This design choice is deliberate; it allows the user to observe the real-time effect of their animation network on the main visuals in the background. This immediate feedback is critical for an intuitive workflow. The bottom panel in this workspace contains the **Audio Panel**, which displays the audio waveform and is synchronized with the custom player, ensuring the playback head on the timeline always matches the currently rendered visual frame.

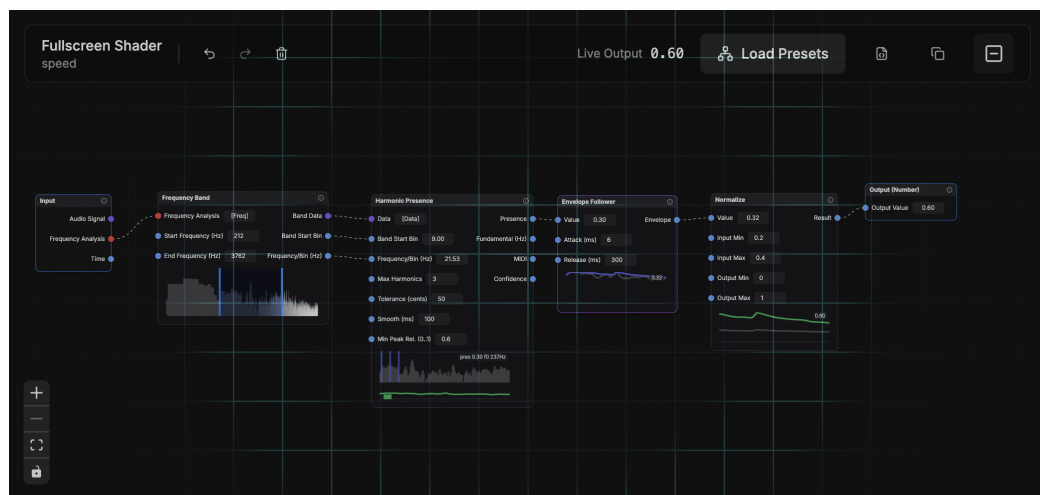


Figure 4.21: The Animation Builder interface, presented as a semi-transparent overlay so users can edit a node network while seeing the visual output in the background.

The integration of these distinct but related tools—layer management, a live preview, and animation editing—into a cohesive, interactive workspace is intended to facilitate an uninterrupted creative process. By minimizing cognitive load and keeping relevant controls and feedback loops in close proximity, the UI design aims to support a more direct and intuitive workflow.

Chapter 5

Implementation

The preceding chapter detailed the architectural blueprint of the VizEngine prototype. This chapter shifts focus from design to practice, documenting the key implementation details and the technological foundation upon which the system is built. We will examine the specific libraries and frameworks that were selected to realize the architectural goals, the design of the developer-facing APIs that ensure the system's extensibility, and the methodologies employed for testing and quality assurance. This chapter describes the "how" of the project's construction, bridging the gap between the conceptual architecture and the tangible, working application.

5.1 Technology Stack

The selection of the technology stack was guided by the core requirements of the project: real-time performance, a dynamic user interface, and an extensible developer framework. We deliberately chose a modern, JavaScript-based ecosystem, which we can categorize into three main areas: the core application framework, the rendering and state management engine, and the supporting interaction libraries.

5.1.1 Core Framework and UI Foundation

Next.js: Application Scaffolding The entire application is built within the **Next.js** [35] framework. While VizEngine currently operates primarily as a client-side application, using Next.js provides a robust foundation for build optimizations and a clear pathway for future enhancements, such as user accounts or server-side rendering of project galleries.

React: The Declarative UI Library **React** [22] was selected for the entire user interface. Its declarative, component-based paradigm was a natural fit for the editor's complex and stateful nature, directly enabling key features like the dynamic generation of UI forms from schemas (see Section 4.6).

5.1.2 Rendering, State, and Timing Engine

Three.js: High-Performance 3D Rendering For all 3D rendering, we selected **Three.js** [21], the de facto standard library for WebGL. Its high-level abstractions, such as the scene graph, and performance features, like instanced rendering, were critical for enabling the dual 2D/3D rendering path.

Zustand: Minimalist State Management For global state management, we chose **Zustand** for its minimalist API and performance-oriented design. Its selector-based subscription model was essential for preventing unnecessary UI re-renders, a critical factor in maintaining a responsive application.

Remotion: Frame-Accurate Timing The **Remotion** library was integrated for one critical function: to serve as a reliable, frame-based clock that acts as the "heartbeat" for the application. It provides the stable `useCurrentFrame` hook that synchronizes all real-time visual updates. The system's video export capabilities are handled by a separate, custom-built pipeline (Section 4.9) and do not use Remotion's rendering or export features.

5.1.3 Supporting Interaction and Component Libraries

To accelerate development and ensure a high-quality user experience, we leveraged several specialized open-source libraries.

Shadcn/ui and Radix UI: Component Toolkit The editor’s UI components are built using **Shadcn/ui** [36], a collection of components built on the accessible primitives from **Radix UI**. This approach provided maximum styling flexibility while ensuring robust and accessible UI elements.

React Flow (XYFlow): Node-Based Editor Canvas The rendering and interaction logic for the node editor is powered by **React Flow** (now **XYFlow**) [37]. This highly customizable library provides the core functionality for rendering nodes and edges on a pannable, zoomable canvas.

dnd kit: Layer Drag-and-Drop Functionality To implement the re-ordering of layers, we used **dnd kit** [38], a modern, lightweight, and accessible toolkit for building drag-and-drop interfaces in React.

WaveSurfer.js: Audio Visualization The audio panel’s waveform display is powered by **WaveSurfer.js** [39]. Its React wrapper handled the decoding and rendering of the waveform, providing an intuitive interface for audio navigation.

FFmpeg.wasm for Video Encoding The custom export pipeline (Section 4.9) is powered by **FFmpeg.wasm** [40]. This WebAssembly port of the entire FFmpeg suite allows for complex video and audio processing to run directly in the user’s browser. We leverage it to take the sequence of rendered frames from IndexedDB and encode them, along with the audio track, into a standard MP4 or WebM video file, enabling a powerful, serverless export solution.

5.2 Extensibility and Developer Experience

The long-term viability and utility of a creative software tool are contingent upon its extensibility. We therefore designed the VizEngine’s architecture to allow developers to easily contribute new visual elements and data processing capabilities. This was achieved by creating a consistent, type-safe, and declarative Application Programming Interface (API) for both visual **Components** and animation **Nodes**. This focus on developer experience (DX) not only simplifies the manual coding process but also creates a predictable structure that is ideal for automation and AI-powered code generation.

5.2.1 Creating a New Visual Component

A **Component** is the fundamental building block of any visual scene. To simplify their creation, we developed a `createComponent` factory function that takes a single definition object. This pattern provides a clear and structured way to package a component’s configuration, logic, and metadata, abstracting away the underlying boilerplate.

This pattern allows a component author to focus on three key areas:

1. **Configuration Schema:** Defining the public parameters using the `v.config()` API, as detailed in Section 4.6. This schema is used to automatically generate the component’s user interface.
2. **Rendering Logic:** Implementing either a `draw` function for 2D visuals or `init3D/draw3D` functions for 3D visuals.
3. **State Management (Optional):** Providing a `createState` function if the component needs to maintain an internal state that persists across frames.

The rendering functions receive a single, well-typed context object containing all necessary data and utilities, such as the layer’s current `config` values, the `audioData` for the frame, and the relevant rendering context (`canvasCtx`

or `threeCtx`). A complete, functional definition is shown in Listing 11. Once a component file is added to a designated directory, the development server automatically discovers, registers, and makes it available in the UI, with full support for hot-reloading.

```
1 import { createComponent, v } from 'viz-engine';
2 import * as THREE from 'three';
3
4 // A complete, functional component definition.
5 export const RotatingCube = createComponent({
6   name: 'Rotating Cube',
7   config: v.config({
8     color: v.color({ defaultValue: '#FF6347' }),
9     rotationSpeed: v.number({ defaultValue: 1, min: -5, max: 5 }),
10  }),
11
12  // Setup runs once when the layer is created.
13  init3D: ({ threeCtx }) => {
14    const geometry = new THREE.BoxGeometry(1, 1, 1);
15    const material = new THREE.MeshStandardMaterial();
16    const cube = new THREE.Mesh(geometry, material);
17
18    // Store the cube in the scene's userData for access in draw3D.
19    threeCtx.scene.userData.cube = cube;
20    threeCtx.scene.add(cube);
21  },
22
23  // draw3D runs every frame.
24  draw3D: ({ threeCtx, config, dt }) => {
25    const { cube } = threeCtx.scene.userData;
26    if (cube) {
27      cube.material.color.set(config.color);
28      cube.rotation.y += config.rotationSpeed * dt;
29    }
30  },
31 });
```

Listing 11: A complete and minimal definition for a new 3D component.

5.2.2 Creating a New Animation Node

The same principle of a declarative, structured API applies to creating new **Nodes** for the animation engine. This allows developers to easily add custom data processing and audio analysis capabilities to the visual programming environment.

The `createNode` factory function requires a definition of the node's `inputs` and `outputs` (including their data types and default values) and a `computeSignal` function that contains the node's core logic, as shown in Listing 12. This functional and type-safe approach makes it straightforward to extend the animation system's vocabulary, enabling the construction of increasingly sophisticated signal processing graphs.

```
1 import { createNode } from 'viz-engine';
2
3 // A node that computes the Root Mean Square (RMS)
4 // of an audio signal.
5 export const RmsNode = createNode({
6   label: 'RMS',
7   description:
8     'Calculates the Root Mean \
9     Square (average power) of a data array.',
10
11   // This node expects an array of numbers (like audio data).
12   inputs: [{ id: 'data', type: 'data-array' }],
13
14   // It outputs a single number.
15   outputs: [{ id: 'rms', type: 'number' }],
16
17   computeSignal: ({ data }) => {
18     if (!data || data.length === 0) {
19       return { rms: 0 };
20     }
21
22     // Standard RMS calculation.
23     const sumOfSquares =
24       data.reduce((sum, value) => sum + value * value, 0);
25     const meanSquare = sumOfSquares / data.length;
26     const rms = Math.sqrt(meanSquare);
27
28     return { rms };
29   },
30 });
```

Listing 12: A complete definition for a custom audio analysis node.

5.2.3 Architecture for AI-Assisted Development

A significant benefit of this structured, declarative API is that it is not only beneficial for human developers but also renders the codebase exceptionally well-suited for **Large Language Model (LLM) assisted code generation**.

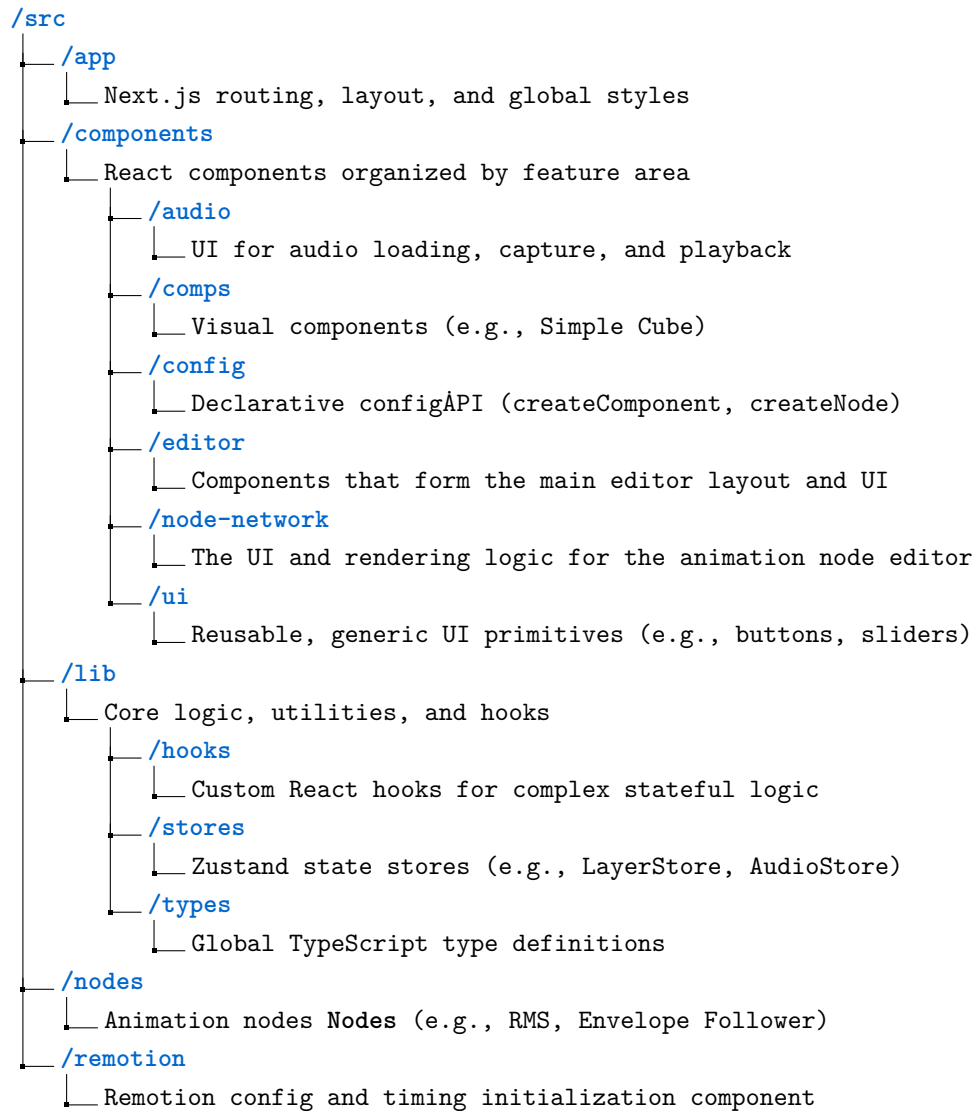
Because the API provides a clear, constrained, and type-safe "template" for both components and nodes, an LLM can be prompted to generate new

functionality with a high degree of success. The model can focus on the novel logic within the rendering or computation functions, while the surrounding boilerplate (configuration, state management, registration) is handled by the consistent API structure. This opens a clear pathway for a future workflow where a developer could describe a visual idea in natural language and have an AI generate a functional, ready-to-use component file, drastically accelerating the prototyping and development process. Informal experiments conducted during this research confirmed this potential; a state-of-the-art LLM model (claude 4.5 sonnet) was consistently able to generate simple 2D and 3D components from a single prompt, and could produce more complex components with iterative feedback guided by the TypeScript compiler.

5.3 Source Code Organization

To ensure the codebase is maintainable and scalable, we adopted a feature-based directory structure within the main `/src` directory. Core architectural domains, such as state stores, visual components, and animation nodes, are organized into their own top-level directories. This organization ensures a clear separation of concerns and makes the codebase intuitive to navigate for future development.

The high-level structure of the application's source code is illustrated below, with a focus on the key architectural directories:



This structure ensures a clear separation of concerns. For example, all global state logic is centralized in `/lib/stores`, while the creative content of the application—the visuals and the animation processing blocks—reside in `/components/comps` and `/nodes`, respectively. This modularity was a key factor in enabling the extensibility patterns described in the previous section.

5.4 Testing and Quality Assurance

Although a comprehensive test suite was not within the scope of this initial prototype, the system was designed from the ground up with **testability** as a core principle. The strong separation of concerns, the use of pure functions for computation, and the declarative nature of components create an architecture that is well-prepared for robust automated testing. This section outlines the pragmatic, multi-layered strategy we adopted for ensuring quality.

5.4.1 A Strategy for a Visual System

A robust quality assurance strategy for a highly visual and interactive application necessitates a multi-layered approach. We identified three primary categories of tests, each serving a distinct purpose in verifying the system's correctness, as detailed in Table 5.1.

Table 5.1: The three primary categories of tests for the VizEngine.

Test Category	Scope and Purpose
Unit Tests	Scope: Individual functions, nodes Purpose: Verify that the smallest pieces of logic behave correctly in isolation.
Integration Tests	Scope: Interactions between modules Purpose: Ensure that different parts of the system work together as expected.
Visual Regression Tests	Scope: Pixel-level output Purpose: Automatically detect unintended visual changes by comparing screenshots against a baseline.

5.4.2 Unit Testing the Core Logic

The most direct application of automated testing within the current architecture is the unit testing of the computational units: the animation nodes. Because the `computeSignal` function of most nodes is a pure function—its output depends only on its inputs—it can be tested without any complex setup or mocking. As shown in Listing 13, we can provide a set of known inputs and assert that the function produces the expected output. This allows us to verify the functional correctness of the core data processing logic with a high degree of confidence.

```
1 import { MathNode } from './nodes/MathNode';
2
3 // A unit test for the MathNode's computation logic.
4 describe('MathNode', () => {
5   it('should correctly perform the "add" operation', () => {
6     // Arrange: Define the inputs for the node.
7     const inputs = { a: 5, b: 10, op: 'add' };
8
9     // Act: Execute the node's core logic.
10    const result = MathNode.computeSignal(inputs);
11
12    // Assert: Check if the output is as expected.
13    expect(result.result).toBe(15);
14  });
15
16   it('should handle division by zero gracefully', () => {
17     const inputs = { a: 5, b: 0, op: 'divide' };
18     const result = MathNode.computeSignal(inputs);
19     expect(result.result).toBe(0);
20   });
21 });
```

Listing 13: A unit test for an animation node, verifying its functional correctness.

5.4.3 Proposed Strategy for Visual Regression Testing

A primary challenge in the quality assurance of any generative graphics application is the validation of the rendered visual output. Manually checking the rendering after every code change is impractical and error-prone. The planned solution for this is **visual regression testing**.

The proposed workflow is as follows:

1. **Create Test Cases:** A suite of test projects would be created, each designed to render a specific component or feature in a deterministic way (e.g., using a pre-recorded audio snippet at a fixed time).
2. **Generate Baseline Images:** A testing script would load each test project, render a specific frame, and save a screenshot of the output canvas. These initial screenshots are manually approved and stored as the "baseline" or "golden" images.
3. **Automated Comparison:** After any future code changes, the testing script is run again. It generates new screenshots and compares them, pixel by pixel, against the corresponding baseline images.
4. **Detect Regressions:** If any pixel differences are detected above a certain threshold, the test fails, immediately alerting the developer to an unintended visual consequence. A human can then review the visual difference, as shown in Figure 5.1, and either approve the new image as the baseline or fix the bug.

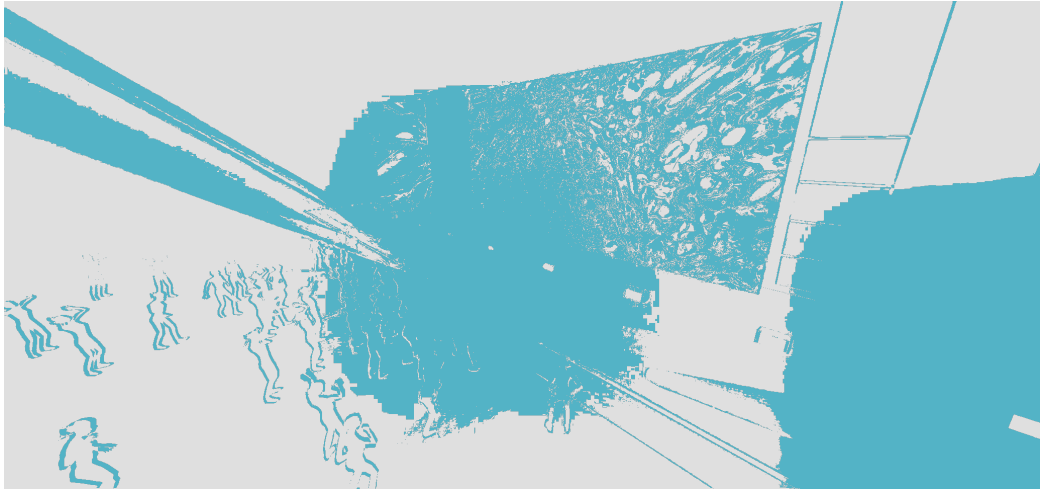


Figure 5.1: An example of a visual diff, highlighting the pixels (in blue) that have changed between a baseline image and a new rendering, allowing for rapid detection of visual bugs.

By automating the process of "looking at the pixels," visual regression testing would provide a powerful safety net, allowing us to extend and refactor the rendering engine with a high degree of confidence. This, combined with unit tests for the core logic, forms a comprehensive and practical quality assurance strategy for the future development of the VizEngine.

Chapter 6

Evaluation

The preceding chapters have detailed the architectural design and implementation of the VizEngine prototype. This chapter provides a critical evaluation of the system to determine how effectively it meets its core objectives. The evaluation is structured to assess the prototype against four key criteria: its **expressive capabilities**, demonstrated through a series of practical compositions; its **technical performance** under a real-time load; its position relative to **existing tools** in the field; and its **usability**, assessed through a qualitative user study.

6.1 Evaluation of Expressive Capabilities

To evaluate the creative potential and flexibility of the VizEngine, we created a series of distinct visual compositions, hereafter referred to as "VizComps". Each composition was designed to test the boundaries of a specific architectural subsystem, from detailed audio analysis in the node network to complex multi-layer 3D rendering.

6.1.1 Composition 1: Real-Time Audio Feature Extraction

Objective The primary objective of this composition was to demonstrate the analytical power and modularity of the node-based animation engine. The goal was to create a simple 2D visualization that deconstructs a piece of electronic music into its core rhythmic and melodic components, displaying the real-time energy of each element as an independent animated bar, as shown in Figure 6.1.

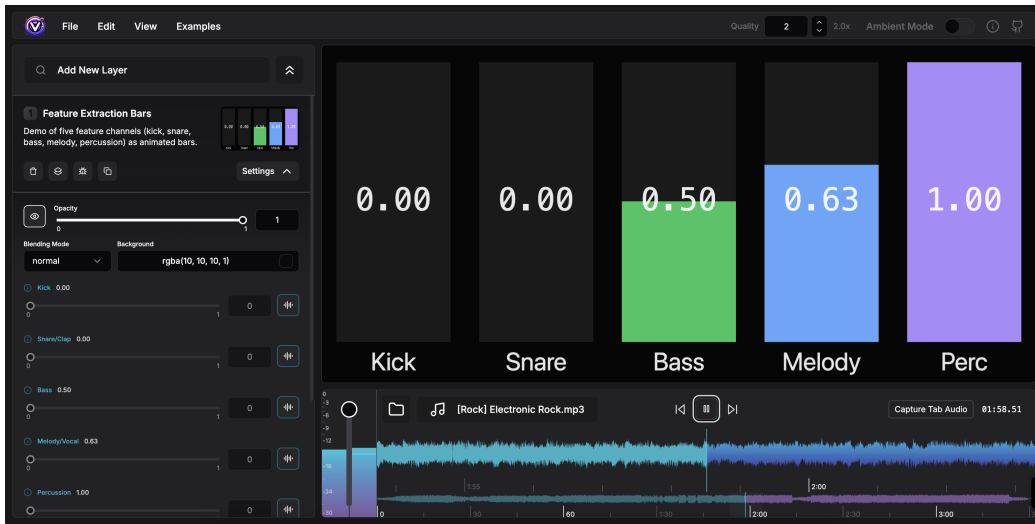


Figure 6.1: The "Feature Extraction Bars" composition, which uses five distinct node networks to drive a 2D visualization of key musical elements.

Implementation This was achieved using a single 2D Canvas component, "Feature Extraction Bars". The component's configuration exposes five numerical parameters: `kick`, `snare`, `bass`, `melody`, and `percussion`. The core of the composition lies in the five sophisticated, pre-built `defaultNetworks` that were designed to animate each of these parameters.

The strategy for each element demonstrates the flexibility of the node graph:

- **Kick and Snare:** For these core percussive elements, the node net-

works were designed to act as transient detectors. They first use a **Frequency Band** node to isolate the characteristic frequencies of the instrument (e.g., 80-150Hz for the kick). The signal is then smoothed with an **Envelope Follower**, normalized to handle changes in song volume with an **Adaptive Normalize** node, and finally passed through a **Hysteresis Gate** node to produce a sharp, binary-like trigger signal. This multi-stage process is highly effective at isolating specific drum hits.

- **Bass:** For sustained bass energy, a simpler network was used, focusing on isolating low-frequency content (20-163Hz) and smoothing it over time to represent the "presence" of the bassline rather than individual notes.
- **Melody:** To detect melodic and vocal content, a more sophisticated approach was required. We utilized a custom **Harmonic Presence** node, which analyzes a frequency band to score the likelihood of tonal content by searching for a harmonic series. This allows the visualization to react specifically to pitched instruments, ignoring noisy or percussive sounds within the same frequency range.
- **Percussion:** To capture high-frequency percussive elements like hi-hats and cymbals, a network similar to the snare's was used, but focused on a high-frequency band (4-10kHz).

Evaluation This composition successfully demonstrates several key capabilities of the system. First, it validates that the node-based animation engine is powerful enough to perform sophisticated, multi-stage audio analysis in real-time. Second, it proves the value of custom, high-level analysis nodes like **Harmonic Presence** for achieving more musically intelligent results. Finally, it showcases the utility of the **defaultNetworks** feature, which allows complex signal processing graphs to be encapsulated and easily reused, providing powerful functionality to end-users without requiring them to build the network

from scratch.

6.1.2 Composition 2: Multi-Layer 2D/3D Compositing

Objective The second composition was designed to evaluate the effectiveness and flexibility of the layer-based compositing system, a core architectural pillar of the VizEngine. The objective was to create a visually rich scene by stacking and blending multiple, disparate layers to test the capabilities of the DOM-based multi-canvas rendering pipeline. This showcase moves beyond a single monolithic visual to demonstrate how independent 2D shaders and complex 3D components can be integrated into a cohesive, audio-reactive whole using hardware-accelerated CSS blending modes, as depicted in Figure 6.2.

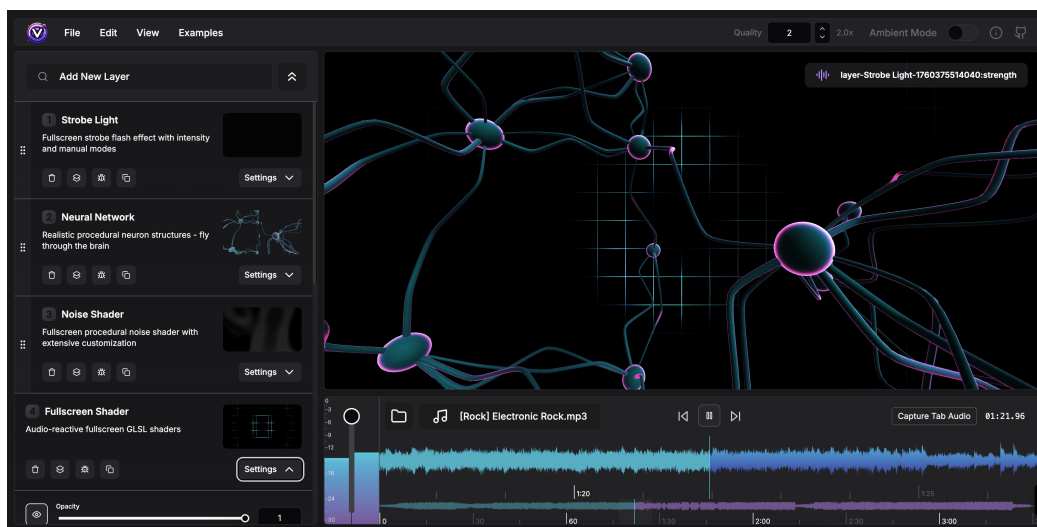


Figure 6.2: The "Layer Blending Showcase" composition, demonstrating the combination of a 3D neural network, 2D shaders, and impact layers using CSS blend modes.

Implementation The composition is constructed from four distinct layers, each serving a specific visual purpose and utilizing a different blending mode to interact with the layers beneath it. This bottom-to-top construction creates

a final image with significant depth and texture.

1. **Layer 1 (Base):** A ‘Fullscreen Shader’ component using a ‘normal’ blend mode provides the foundational visual element—an animated, teal-colored "Radial Ripple Grid." It establishes the scene’s base color palette and motion.
2. **Layer 2 (Texture):** A ‘Noise Shader’ component, configured to produce an inverted, high-contrast Perlin noise pattern resembling white smoke, is layered on top using a ‘**multiply**’ blend mode. This is a key feature: the white areas of the smoke leave the underlying ripple grid unaffected, while the black areas darken it, adding a layer of organic texture and shadow.
3. **Layer 3 (Focal Point):** A 3D ‘Neural Network’ component is rendered with a **transparent background** and ‘normal’ blending. This allows the complex, procedurally generated 3D structure—with its glowing cyan dendrites and pink signal orbs—to appear as if it is floating within the textured 2D environment created by the first two layers. This layer contains its own audio-reactivity, with kick drums triggering neural signals and snare hits periodically regenerating the network structure.
4. **Layer 4 (Impact):** A simple ‘Strobe Light’ component is placed at the top of the stack. It uses a ‘**soft-light**’ blend mode, which causes the full-screen white flashes to brighten the entire scene in a subtle, cinematic manner, increasing contrast without washing out the colors of the layers below. The strobe’s flash rate and brightness are driven by the combined intensity of the kick and bass frequencies.

Evaluation This composition successfully validates several critical architectural decisions of the VizEngine. Primarily, it proves the viability and power of the **DOM-based multi-canvas rendering pipeline**. By delegating the final compositing task to the browser’s rendering engine via standard

CSS properties like ‘mix-blend-mode’, the system leverages highly optimized, hardware-accelerated blending at no additional JavaScript cost. This approach is both performant and creatively expressive.

Furthermore, the composition demonstrates the seamless integration of **2D and 3D rendering contexts within a single scene**. The ability to layer a Three.js scene (Layer 3) between two 2D canvas shader layers (Layers 2 and 4) without conflict showcases the engine’s versatility and the robustness of the layer isolation model.

Finally, the showcase proves that the layer system is more than a technical feature; it is a powerful creative paradigm. It allows users to achieve complex visual effects—such as textural overlays and non-destructive lighting—in a declarative and intuitive way, enabling the construction of intricate, evolving scenes by combining simple, reusable, and independent visual components.

6.1.3 Composition 3: Complex Real-Time 3D Scene

Objective The final composition was designed to serve as a comprehensive stress test for the entire VizEngine architecture. The objective was to move beyond simple primitives and evaluate the system’s capability to render and manage a professional-grade, navigable 3D scene with a high degree of complexity. This composition, an electronic dance music (EDM) stage, shown in Figure 6.3, was created to test the synergy and performance of all core subsystems—the Three.js rendering pipeline, the state management architecture, and the node-based animation engine—under a significant and sustained real-time load.

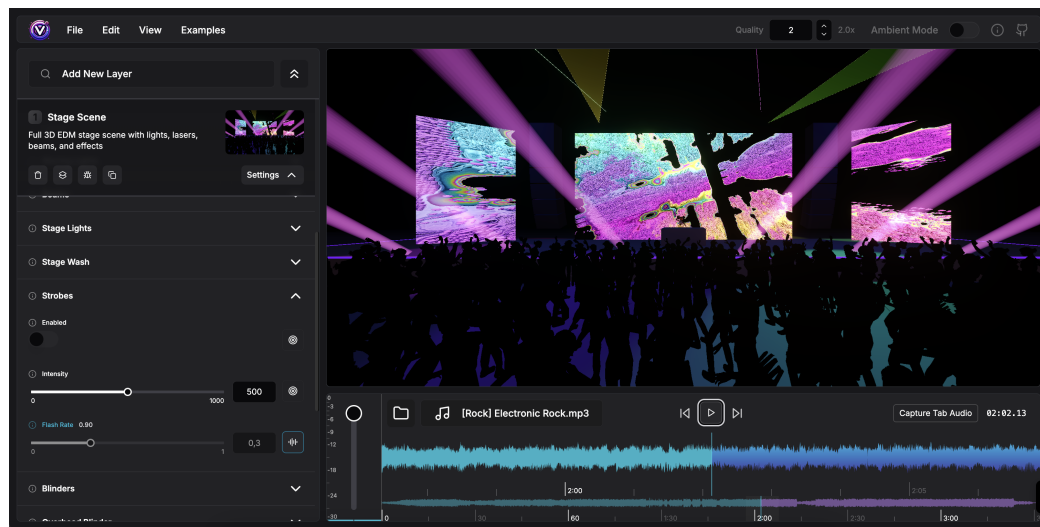


Figure 6.3: The "Stage Scene" composition, a complete 3D environment designed to test the performance limits and expressive capabilities of the VizEngine.

Implementation and Technical Deep Dive The scene was implemented as a single, highly complex visual Component that encapsulates an entire virtual concert environment. Its construction serves as a practical case study for the engine's architectural strengths, integrating numerous advanced graphical, interactive, and audio-reactive features.

Case Study 1: GPU-Accelerated Crowd Rendering A primary performance challenge was rendering a large, animated crowd. A naive approach of managing hundreds of individual animated models on the CPU would immediately cripple performance. To solve this, we implemented **GPU-accelerated instanced skeletal animation**.

- **Technique:** The system uses a custom 'InstancedMesh' capable of rendering up to 1,000 characters in a single draw call. The skeletal animation data for every instance is pre-computed each frame and baked into a data texture, often called a "bone texture."

- **Implementation:** A custom GLSL vertex shader reads the appropriate bone transformation matrices from this texture for each vertex of each instance, performing the final skinning deformation entirely on the GPU. This offloads the most intensive part of the animation process from the CPU, a technique critical for rendering large-scale animated crowds in real-time [41].
- **Architectural Proof:** This demonstrates that the component architecture is flexible enough to accommodate highly specialized, performance-critical rendering techniques that require custom shaders and low-level data management, proving its suitability for demanding visualization tasks.

Case Study 2: Procedural Generation via Raymarching The main video wall is not a pre-rendered video but a live visualization. This was achieved by integrating custom shader logic directly into a Three.js material.

- **Technique:** The effect is generated by a GLSL fragment shader that uses **raymarching of a signed distance field (SDF)** to render an infinite Mandelbox fractal shape. This purely mathematical approach allows for infinite detail and fluid animation without relying on texture memory or video decoding.
- **Implementation:** The component’s ‘draw3D’ method passes time and audio-reactive parameters (e.g., ‘scale’, ‘rotationSpeed’, ‘travelSpeed’) as uniforms to the shader on every frame. This allows the fractal’s properties to be directly manipulated by the node-based animation engine. The technique is heavily inspired by the demoscene and real-time graphics research [42].
- **Architectural Proof:** This validates the engine’s ability to seamlessly bridge its high-level node graph with low-level, custom GLSL code, empowering developers to create deeply procedural and computationally intensive visuals.

Case Study 3: Nuanced Audio-Reactive Signal Chains The composition’s 13 default node networks highlight the expressive power of the animation engine beyond simple amplitude-to-scale mappings. Two examples demonstrate this depth:

- **The ‘overhead-blinder-big-impact’ Preset:** This network is designed to trigger a powerful overhead light only on the most significant musical drops. Its signal chain consists of: ‘Frequency Band (20-200Hz)’ → ‘Adaptive Normalize (Quantile: 0.98)’ → ‘Hysteresis Gate (Threshold: 0.87-0.96)’. The extremely high quantile and gate thresholds ensure that only the top 2% of bass energy events over a long time window will trigger the effect. This demonstrates the creation of highly specific, musically-aware event detectors.
- **The ‘beam-mode-melody-cycle’ Preset:** This network changes the visual pattern of the beam lights in response to the bassline. Its chain uses a ‘Threshold Counter’ node rate-limited to 500ms. Rather than reacting to every transient, it detects significant changes in the bass melody’s presence and cycles through a predefined list of visual modes. This demonstrates a higher level of structural analysis, reacting to melodic phrases rather than individual notes.

These complex signal chains, built visually without writing code, validate the core premise of the thesis: that the node-based paradigm provides an accessible yet powerful interface for designing sophisticated, musically intelligent behaviors.

Evaluation This composition successfully validates the VizEngine’s architecture as being capable of handling tasks that approach the complexity of professional, desktop-based visualization software.

First, it proves that the **rendering pipeline and state management are performant under pressure**. Simultaneously managing hundreds of configurable parameters, running 13 parallel node networks, updating

thousands of instanced character animations, and rendering multiple lighting systems and post-processing effects at a stable frame rate confirms the efficiency of the underlying architectural choices.

Second, it serves as a powerful demonstration of **architectural synergy**. This single component utilizes every major feature of the engine: the declarative configuration system for its vast array of parameters, the multi-canvas architecture for post-processing effects like bloom [43], the state management stores for handling camera modes and dynamic updates, and the node engine for its deep audio-reactivity.

Ultimately, this composition confirms that the VizEngine prototype is not merely a tool for creating simple, isolated visuals, but a robust and scalable platform capable of creating immersive, complex, and deeply musical real-time 3D experiences. It successfully meets its objective as a stress test and serves as a compelling proof-of-concept for the viability of the web as a platform for professional-grade, audio-driven visualization.

6.2 Performance Evaluation

While expressive capability is a primary goal, a real-time system is only viable if it performs adequately. This section provides a quantitative, comparative evaluation of the VizEngine prototype's performance. To move beyond high-level browser monitoring, a custom, in-engine profiler was developed. This tool provides granular data on all subsystems, from main thread timing to the computation cost of individual animation nodes, allowing for a deep analysis of the architecture's real-world efficiency and bottlenecks.

6.2.1 Methodology

To validate the "web-native" claim of the architecture, a comparative methodology was used. The exact same complex test composition was profiled for 60 seconds in a production build on two distinct hardware platforms. This test scene was the **"Layer Blending Showcase"** detailed in the previous chapter

(Section 6.1.2, Figure 6.2). As described, this composition is a representative and demanding workload, consisting of four layers (three 2D shaders and one complex 3D component) and five active node networks, stressing all major subsystems of the engine. Detailed explanations of how each metric is calculated by the custom profiler can be found in Appendix B.

To specifically isolate and measure the performance of the core rendering pipeline and the node evaluation engine under sustained load, the 60-second profiling sessions were conducted without active user interaction within the editor interface (e.g., mouse movements, node graph manipulation, UI updates). It should be noted that real-world usage involving significant UI interaction will naturally impose additional computational load on the main thread, potentially resulting in lower effective frame rates during intensive editing operations compared to the baseline rendering performance measured here.

The two test platforms were:

- **Platform 1 (macOS):** Apple MacBook Pro with an M1 Pro CPU, 16GB RAM, and an integrated Apple GPU (running via ANGLE/Metal).
- **Platform 2 (Windows):** Razer laptop with an Intel i7-10750H CPU, 16GB RAM, and a discrete NVIDIA GeForce RTX 2060 GPU (running via ANGLE/D3D11).

A critical factor in this comparison is the difference in rendering load. As detailed in Table 6.1, both tests were run with a 2x resolution multiplier. The macOS platform rendered at **3024x1964 (5.9 million pixels)** due to its native display resolution. The Windows platform, with a base resolution of 1920x1080 and a 2x multiplier, rendered at **3840x2160 (8.3 million pixels)**. The Windows/RTX 2060 platform was therefore rendering approximately **40% more pixels** per frame.

6.2.2 Comparative Results

The key findings from the two 60-second profiling sessions are summarized in Table 6.1. The complete performance data for both platforms is visualized in Figures 6.4 and 6.5.

Table 6.1: Comparative performance metrics for the "Layer Blending Showcase" (60s session).

Metric	Platform 1 (macOS / M1 Pro)	Platform 2 (Windows / RTX 2060)
Effective Resolution	3024 x 1964 (5.9M pixels)	3840 x 2160 (8.3M pixels)
Mean FPS	111.6 FPS	84.4 FPS
Mean Frame Time	9.02 ms	12.03 ms
Max Frame Time (Worst Spike)	73.5 ms	99.4 ms
Frame Stability Score	88.7 / 100	84.8 / 100
Mean Node Network Compute Time	<0.17 ms (per network)	<0.19 ms (per network)
‘Neural Network‘ Avg. Render	3.63 ms	5.22 ms
‘Neural Network‘ Max Render	20.17 ms	19.43 ms
P95 JS Heap Size	176.8 MB	187.1 MB



Figure 6.4: Performance Profile on Platform 1 (Apple M1 Pro). Top: FPS over time. Middle: Memory Usage (L) and Frame Budget Usage (R). Bottom: Layer Render Time (L) and Node Network Compute Times (R).

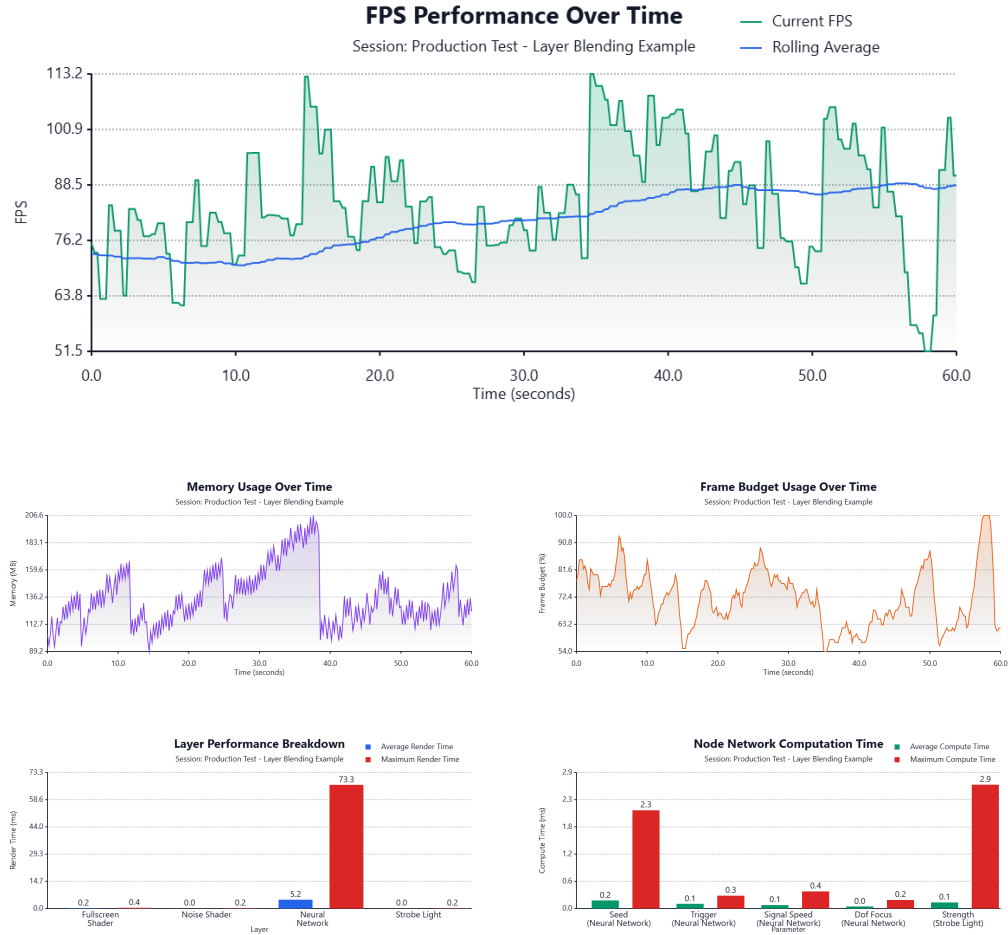


Figure 6.5: Performance Profile on Platform 2 (Windows / RTX 2060). Top: FPS over time. Middle: Memory Usage (L) and Frame Budget Usage (R). Bottom: Layer Render Time (L) and Node Network Compute Times (R).

6.2.3 Analysis of Results

The comparative analysis provides strong validation for the system's architecture and reveals several key findings about cross-platform web performance.

Overall Performance Both platforms delivered excellent performance well above the 60 FPS target, even while rendering millions of pixels. The macOS platform (5.9M pixels) achieved a mean frame time of 9.02ms (111.6 FPS), while the Windows platform, despite rendering 40% more pixels (8.3M), was also highly performant with a 12.03ms mean frame time (84.4 FPS). This demonstrates that the core engine is highly efficient and capable of running complex scenes on modern, disparate hardware.

Validation 1: Node Engine Performance A primary thesis goal was to ensure the node-based animation engine was not a bottleneck. The data from both platforms definitively validates this architecture. As shown in the "Node Network" charts (Figures 6.4 and 6.5), the **average computation time for all networks was sub-millisecond on both systems**. Even the maximum compute spikes (e.g., 1.6ms on Mac, 2.9ms on Windows) were trivial. This proves that the dataflow evaluation system is highly efficient and not a performance concern on either architecture.

Validation 2: Rendering and Bottleneck Identification Both platforms confirm that the system's performance is not limited by the core engine, but by the **computational cost of the visual components**. The 2D shader layers (e.g., 'Noise Shader') were trivial on both machines, with average render times under 0.1ms.

The 'Neural Network' 3D layer was correctly identified as the primary source of computational load on both systems. The data shows a logical correlation between pixel count and render time: the Windows platform (8.3M pixels) had a slightly higher average render time (5.22ms) than the Mac (5.9M pixels, 3.63ms). More interestingly, the **maximum render spike was nearly identical on both platforms** (20.17ms vs. 19.43ms). This strongly suggests a specific, expensive operation (likely geometry regeneration) that is compute-bound, not pixel-bound, and is equally costly on both GPU architectures.

Stability and "Perfect Storm" Spikes While average performance was high, the profiler successfully captured the true worst-case frame drops. Both platforms experienced a significant main-thread spike that was far greater than the worst render spike: **73.5ms on macOS** and **99.4ms on Windows**. This is a critical and realistic finding. It demonstrates that the main threat to stability is a "perfect storm" event, where a maximum render spike, a compute spike, and (most likely) a major JavaScript Garbage Collection event (visible in the sawtooth "Memory Usage" charts) all occur within the same frame.

6.2.4 Future Optimization Pathways

The performance analysis provides a clear, data-driven roadmap for future optimizations. While the current single-threaded architecture performs well on average, its vulnerability to these "perfect storm" spikes highlights several key opportunities.

- **Offloading to Web Workers:** The most significant planned enhancement is to move the entire node network computation into a dedicated **Web Worker**. This would prevent computational spikes, even small ones like the 2.9ms node spike on Windows, from contributing to a main-thread blockage.
- **GPU Computation (GPGPU):** For visually complex components like the 'Neural Network', which showed a ~20ms render spike on both platforms, advanced techniques like **WebGL compute shaders** could be used to move expensive logic (e.g., physics, particle updates, geometry generation) from the CPU/JS main thread directly onto the GPU.
- **WebAssembly (Wasm) for Critical Nodes:** For computationally intensive nodes that must run frequently, the core logic could be rewritten in a language like Rust or C++ and compiled to **WebAssembly** for near-native performance.

These pathways demonstrate that the current architectural foundation is not only performant for its intended scope but is also well-suited for scalable, future enhancements.

6.3 Comparative Analysis

To provide context for the contributions of this thesis, it is necessary to compare the VizEngine prototype to established solutions in the field of real-time visual programming and creative coding. While a wide range of tools exists, we have selected two key systems for direct comparison, each representing a different segment of the market: **TouchDesigner**, a professional, desktop-based industry standard, and **cables.gl**, a contemporary, web-native creative platform.

6.3.1 Methodology

The comparison is based on a set of five criteria that are directly relevant to the stated goals of the VizEngine project:

1. **Platform:** The underlying technology and operating environment (e.g., desktop application vs. web-based).
2. **Accessibility:** A measure of how easily a new user can access and begin using the tool, considering both cost (licensing model) and openness (open-source vs. proprietary).
3. **Primary Paradigm:** The core creative model the tool is built upon.
4. **Extensibility:** The primary method by which a developer can add new functionality to the system.
5. **Target Workflow:** The primary use case or creative workflow the tool is optimized for.

6.3.2 Comparison

The results of the comparative analysis based on the above criteria are presented in Table 6.2.

Table 6.2: A comparative analysis of VizEngine against established creative tools.

Platform	
System	Description
VizEngine (Prototype)	Web-based (Browser)
TouchDesigner [2]	Desktop Application (Windows/macOS)
cables.gl [25]	Web-based (Browser)
Accessibility	
System	Description
VizEngine (Prototype)	High: Fully open-source (MIT License) and free to use.
TouchDesigner [2]	Low: Proprietary, with a free non-commercial version but requiring a paid license for professional use.
cables.gl [25]	Medium: A freemium model with a free tier and paid subscriptions for advanced features and private projects.
Primary Paradigm	
System	Description
VizEngine (Prototype)	Node-based animation engine combined with a layer-based compositing system.
TouchDesigner [2]	Primarily a node-based, dataflow programming environment.
cables.gl [25]	Primarily a node-based visual programming environment.
Extensibility	
System	Description
VizEngine (Prototype)	High: Developers can add new Components and Nodes via a declarative JavaScript/TypeScript API.
TouchDesigner [2]	High: Highly extensible through Python scripting and C++ for custom operator development.
cables.gl [25]	Medium: Users can create custom operators (“ops”), but the core platform is not open-source.
Target Workflow	
System	Description
VizEngine (Prototype)	Focused specifically on creating shareable, web-native, audio-reactive animations.
TouchDesigner [2]	Broadly focused on live events, interactive installations, and large-scale media systems.
cables.gl [25]	Focused on creating interactive web content, 3D graphics, and shareable web experiments.

6.3.3 Analysis

The comparative analysis in Table 6.2 highlights the specific niche that the VizEngine prototype was designed to fill. While it shares the powerful node-based paradigm with both TouchDesigner and cables.gl, its key differentiators lie in its accessibility and its focused application domain.

Unlike TouchDesigner, VizEngine is designed for a **web-native workflow**, enabling the creation of visuals that can be easily shared and embedded across the web without requiring specialized software. Unlike cables.gl, which is a powerful general-purpose web graphics tool, VizEngine is differentiated by its deep and specific focus on **audio-reactivity** as a first-class citizen, evidenced by its dedicated audio analysis pipeline and core library of audio-processing nodes.

Most significantly, the VizEngine’s status as a fully **open-source** project distinguishes it from the proprietary and freemium models of the established tools. This positions the prototype not as a direct competitor, but as an accessible, community-driven alternative for developers and artists who require a transparent and freely extensible platform specifically for creating audio-driven visuals on the web. The evaluation confirms that the prototype successfully occupies this identified gap in the existing landscape of creative coding tools.

6.4 Usability Study

The final component of our evaluation was a qualitative usability study designed to assess the intuitiveness of the VizEngine’s core creative workflow. While quantitative metrics can validate technical viability, direct observation is necessary to evaluate the user experience of the visual programming paradigm itself, especially for a technical audience.

6.4.1 Methodology

To gather valuable qualitative feedback, we conducted a small-scale, task-based usability study.

Participant Profile The study was conducted with a panel of 5 participants, all male professional software developers (primarily in web development) aged 26-29 years old with university backgrounds in mathematics and computer science. The key selection criterion was that while they possess deep technical expertise, they have little to no prior professional experience with node-based visual programming environments (e.g., TouchDesigner) or digital art software. This profile allows us to specifically evaluate whether the visual dataflow paradigm offered by the VizEngine serves as an intuitive abstraction for programmers accustomed to text-based logic.

Study Protocol Each participant underwent an individual session lasting approximately 25 minutes, following a structured, “think-aloud” protocol where they were encouraged to vocalize their thought process. This process was conducted iteratively: minor usability issues identified in one session were often rectified before the next, allowing for a broader range of feedback to be collected. The procedure for each session was as follows:

1. **Pre-Session Briefing (5 minutes):** The participant was introduced to the study’s purpose and the think-aloud method.
2. **Guided UI Tour (5 minutes):** The facilitator provided a brief, high-level tour of the editor’s main panels (Layer Panel, Canvas, Animation Builder, Audio Panel), explaining their purpose without demonstrating the specific steps of the assigned task.
3. **Task Assignment:** The participant was given a clear and representative task: *“Using the provided audio file, create a visual where a simple shape, like a circle or square, changes its size in reaction to the overall volume of the music.”*

4. **Observation Period (15 minutes):** The participant was given up to 15 minutes to complete the task while the facilitator observed and took notes on their workflow, points of hesitation, and verbal feedback.

Evaluation Metrics To structure the observational data, we focused on two types of metrics:

- **Quantitative Metric (Task Completion Rate):** A binary measure of whether the participant successfully completed the core task within the allotted time.
- **Qualitative Metrics (Observational Notes):** The primary data from the study. We recorded specific “sticking points,” moments of insight, and subjective feedback related to how the visual paradigm aligned with or differed from their existing mental models of programming.

6.4.2 Results

The study achieved a **100% task completion rate**, as all five participants successfully created the requested audio-reactive animation. The primary value of the study, however, was the collection of qualitative feedback. These findings can be synthesized into four primary themes.

1. Validation of Core Creative Concepts Participants consistently demonstrated a clear grasp of the VizEngine’s core paradigms. The layer-based compositing system was found to be intuitive; one participant, while creating a masking effect, remarked, "Aha, I understand, that’s how the stack works... It’s a stack, literally." Another noted that the blending mode controls helped them "finally understand" the concept. The core audio-reactive workflow was also well-received, with participants appreciating the live data previews on node connections and the system’s ability to react to live system audio. The node-based paradigm itself was identified as a key strength, with one participant calling the audio-to-parameter connection "ultra legit."

2. Key Usability Challenges in the Node Editor While conceptually praised, the node editor was the source of the most significant user friction.

- **Discoverability:** The workflow for accessing the editor was a significant barrier for some new users. One participant was completely blocked, asking, "I just wanted to get to that node editor. And where is it?," not realizing it is opened by clicking a parameter's "animate" icon.
- **Interaction:** Participants with prior node-based experience found basic interactions to be inefficient. The most-cited issue was the inability to move an existing connection to a new input, which forced users to delete and recreate the connection.
- **Navigation:** Mouse-and-keyboard navigation for panning and zooming was reported as non-standard, creating difficulty for users without a trackpad.
- **Context:** One participant reported becoming "lost" when editing multiple node graphs, as the UI did not clearly indicate which parameter's graph was currently active. This led to a key user suggestion for a **tabbed interface** to manage different animation networks.

3. System-Wide UI Discoverability Issues Beyond the node editor, participants frequently struggled to locate key UI elements.

- **Hidden Features:** Elements such as the "Presets" dropdown were described as "hidden" and not "visible that this is an interactive control." The "Add Layer" button's placement led to it being repeatedly confused with a nearby "Expand All" icon.
- **Confusing Defaults:** The default solid black background for 3D components was cited as counter-intuitive for a layering system, with a user suggesting they should be transparent by default to facilitate composition.

- **Confusing Terminology:** The "File > Init" menu item was universally misunderstood by participants, who agreed it should be renamed to "New Project" or similar.

4. Application Stability and Missing Features The study also identified several bugs and missing quality-of-life features indicative of the system's prototype status.

- **Identified Bugs:** Participants encountered a non-functional "Init" (reset) button, at least one session-ending crash, and a workflow bug where the UI remained unlocked during video export, allowing user interactions to be captured in the final render.
- **Missing Quality-of-Life Features:** The absence of a standard **Undo/Redo** function was a notable point of friction. Participants also expected, but did not find, a functional *Delete* key for nodes, a simple method to reset sliders to their default values, and more robust timeline selection controls.

6.4.3 Analysis and Conclusion

The usability study confirms that the VizEngine's core conceptual model is sound and intuitive for its target technical audience, even as the prototype's implementation revealed several areas for refinement. The 100% task completion rate suggests that the fundamental workflow is logical, allowing users to succeed despite the identified sources of friction. The positive feedback on the node paradigm and the participants' "aha" moments with core concepts validate that the hybrid architecture is effective.

A key insight is that the **node editor, while conceptually praised, was the primary source of interaction friction**. This does not suggest a flaw in the node-based approach itself; rather, it indicates that the prototype's interaction design did not yet meet the high expectations set by other professional creative tools. The feedback on discoverability was also

consistent, indicating that the UI design is not always intuitive for new users, as it relies on assumed knowledge rather than clear, self-explanatory controls.

It is important to note the value of the **iterative study methodology** employed. By addressing minor bugs and usability issues as they were identified, the study was able to progress beyond repeatedly finding the same initial barriers. This evolving-prototype approach not only validated the architecture’s flexibility for rapid changes but also allowed the study to gather a much broader range of feedback on more complex interaction patterns.

The identified bugs and missing features are characteristic of an early-stage prototype, but the feedback is highly actionable. Suggestions from participants—such as a **tabbed node editor**, **drag-to-rewire functionality**, and **transparent-by-default layers**—are not vague complaints but concrete, evidence-based design recommendations. In conclusion, the study successfully validated the project’s core hypothesis and provided a clear, prioritized roadmap for maturing the VizEngine from a functional prototype into a more polished and robust application.

Chapter 7

Conclusion

This thesis has documented the design, implementation, and evaluation of the VizEngine, a prototype for a novel, web-based tool for creating customizable, audio-reactive visualizations. The primary goal of this work was to address a gap between complex, proprietary desktop software and code-intensive creative frameworks by architecting an accessible, extensible, and powerful engine built entirely on modern web technologies. Through a hybrid architecture combining a layer-based compositor with a declarative, node-based animation engine, the project sought to demonstrate the viability of the browser as a platform for real-time creative tooling. This final chapter summarizes the key contributions of this work, acknowledges its limitations, and outlines promising directions for future research and development.

7.1 Summary of Contributions

The primary contributions of this thesis can be summarized in three key areas: a novel system architecture, a successful prototype implementation, and a comprehensive evaluation of its capabilities.

A Hybrid Architecture for Web-Based Visuals We designed and presented a novel software architecture that successfully integrates two distinct

paradigms—a layer-based compositing system and a node-based animation engine—into a cohesive whole. We demonstrated how a declarative, schema-driven system can be used to dynamically generate user interfaces and manage state, and how a multi-store state management pattern can ensure a responsive user experience. The design of the dual-path audio pipeline and the DOM-based, multi-canvas rendering system represents a robust and performant architectural foundation for real-time graphics in a browser environment.

A Working Prototype: VizEngine We successfully implemented this architecture as a tangible, working prototype, VizEngine. This implementation serves as a proof-of-concept, demonstrating that the chosen technology stack—including React, Next.js, Three.js, and the Web Audio API—is not only capable of handling the demands of real-time audio analysis and 2D/3D rendering, but can be integrated into a powerful and user-friendly application. The development of a declarative API for creating new Components and Nodes further validates the extensibility of the design.

A Multi-faceted Evaluation of the System Finally, we presented a comprehensive evaluation of the prototype. The showcase of expressive capabilities demonstrated the system’s potential for creating a wide range of sophisticated audio-reactive visuals, from detailed feature extraction to complex 3D scenes. Crucially, a detailed, cross-platform performance evaluation, using a custom in-engine profiler, provided quantitative validation of the architecture’s efficiency. The results demonstrated high average frame rates (well above 60 FPS) on disparate hardware, confirmed the sub-millisecond efficiency of the core node engine, and identified specific component-level rendering costs and main-thread contention spikes under load as the primary performance bottlenecks. Furthermore, the comparative analysis clearly positioned VizEngine within the existing landscape of creative tools, while the usability study validated the intuitiveness of the user workflow for its target audience.

7.2 Limitations

A critical part of any research is the honest acknowledgment of a project’s limitations. As a prototype, the VizEngine successfully demonstrates its core architectural concepts, but several areas would require significant further development to mature it into a production-ready application.

The most significant limitation is the *single-threaded architecture*. While average performance was demonstrated to be high (Section 6.2), all animation and rendering computations currently run on the main browser thread. The performance analysis explicitly revealed this vulnerability, identifying worst-case frame time spikes (up to 99.4ms on the Windows platform) caused by the confluence of rendering load, node computation, and (probably) JavaScript garbage collection occurring simultaneously on the main thread (Table 6.1). This confirms that under heavy load, particularly during these contention spikes, the responsiveness of the user interface could be degraded.

Secondly, while the system was designed with testability in mind, a *comprehensive, automated test suite* has not yet been implemented. The current lack of a visual regression testing pipeline, in particular, means that refactoring the rendering engine carries the risk of introducing unintended visual defects.

Finally, the initial *library of visual components and animation nodes* is limited. While the framework for extensibility is robust, a much larger and more diverse library would be needed to cater to a wider range of artistic use cases out-of-the-box.

7.3 Future Work

The successful implementation and evaluation of the VizEngine prototype have not only validated its core architecture but have also illuminated a clear and ambitious roadmap for future development. The system’s modularity and extensibility provide a strong foundation upon which a professional-grade, collaborative creative suite can be built. The vision for this future work can

be categorized into several key areas of innovation.

Core Engine and Performance Scaling The performance analysis (Section 6.2) provides clear directions for enhancing scalability. To mitigate the identified main-thread spikes (**up to 99.4ms**), the most impactful step would be moving the node network evaluation to a dedicated **Web Worker**, isolating it from rendering. For specific, computationally heavy nodes (like the $\approx 2.9\text{ms}$ spikes observed), core logic could be rewritten in **WebAssembly (Wasm)**. Furthermore, to address rendering bottlenecks (like the $\approx 20\text{ms}$ ‘Neural Network’ spike), leveraging **GPU compute shaders (GPGPU)** or adopting an off-screen framebuffer architecture (inspired by the export pipeline) for advanced post-processing would be key priorities.

Furthermore, the custom export pipeline (Section 4.9) has already proven the viability of a manual, off-screen framebuffer compositing engine. A key future enhancement would be to adapt this export-focused architecture for the *real-time* renderer. By migrating all layers to render to off-screen buffers on every frame, **potentially leveraging Web Workers and the OffscreenCanvas API to move rendering command execution off the main thread**, we could achieve two significant benefits. Firstly, it would further **decouple rendering workload from the main UI thread**, contributing to improved responsiveness, especially during complex scene rendering or garbage collection events. Secondly, it would unlock the capability for professional-grade **post-processing effects** (such as bloom, depth of field, and chromatic aberration) that operate on the final composited image before it is drawn to the screen.

Expansion of Creative Capabilities The next step in empowering artists would be to dramatically expand the creative vocabulary of the engine. This includes building a far more comprehensive library of advanced **audio analysis nodes** for tasks like robust BPM and beat detection by integrating libraries such as Essentia.js. We also envision a complementary **keyframe-based animation system** to allow for manually authored animations that

can be blended with the procedural, audio-reactive values from the node graph. This, combined with support for more complex layer types, such as a rich text editor and a 2D FX layer, would greatly broaden the system's artistic range.

Workflow and User Experience Refinements The usability study detailed in Section 6.4 provided a clear, actionable roadmap for improving the user workflow, which was a primary goal of the iterative development process. As a direct result of this feedback, **many of the key usability issues identified during the study have already been addressed** in the current version of the prototype. This includes renaming confusing terminology (e.g., "Init" is now "New Project"), fixing critical bugs related to the export process, and implementing a robust, multi-level **undo/redo history**. Future work will continue to address the remaining feedback, such as improving node editor navigation and implementing a "meta-node" system for collapsing complex graphs. This iterative refinement demonstrates the architecture's flexibility and the tangible value of formative user evaluation.

AI-Assisted Development and Generative Content A visionary direction for the VizEngine is the integration of modern AI models to accelerate the creative process. We propose an **in-editor AI assistant**, an LLM-powered chat that could generate new visual Components or animation Nodes from natural language prompts within a sandboxed environment. Furthermore, a new "Generative AI" layer type could be created, allowing users to leverage models like Stable Diffusion to generate novel images and video loops directly within their compositions, truly merging the worlds of procedural generation and artificial intelligence.

The Evolution into a Collaborative Ecosystem The ultimate goal is to transform the VizEngine from a standalone tool into a connected, collaborative platform. The first step is to decouple the core engine and publish it as a standalone **NPM package**, allowing developers to embed

the renderer in any React application. The next major step is to build a **cloud-based backend**, enabling user accounts, cloud project storage, and most importantly, **real-time collaboration**. This would allow multiple users to work on a single project simultaneously. This vision extends to integrating with the physical world, with support for protocols like **DMX and Art-Net** to control real-world stage lighting. By building this ecosystem, the VizEngine has the potential to become not just a tool for individual artists, but a shared, web-native platform for a new generation of collaborative, audio-visual art.

7.4 Final Remarks

This thesis set out to explore whether the modern web browser could serve as the foundation for a truly powerful and accessible tool for creating audio-reactive visuals. The journey has taken us from a review of core signal processing principles, through the detailed design of a hybrid architecture, to the implementation and evaluation of a tangible prototype.

VizEngine, as it stands, is more than a collection of features; it is a compelling demonstration that the browser, empowered by technologies such as WebGL, the Web Audio API, and advanced JavaScript frameworks, has matured into a viable platform for real-time creative applications. By presenting a robust architecture, implementing a functional prototype, demonstrating its strong performance and efficiency across different hardware platforms, and outlining a clear vision for its future, this work concludes that the creation of open-source, web-native, and highly capable creative tools is not only possible but represents a significant and promising direction for the future of digital art on the web.

Appendix A

VizEngine Project Resources

The VizEngine prototype developed for this thesis is fully open-source and publicly available. The complete implementation, including all source code, documentation, and example projects, can be accessed through the following resources:

A.1 Source Code Repository

The entire codebase is hosted on GitHub under the MIT License, allowing free use, modification, and distribution:

`https://github.com/vucinatim/viz-engine`

The repository includes:

- Complete TypeScript/React implementation
- All visual components and animation nodes
- Architectural documentation
- Example projects (.vizengine.json files)
- Setup and development instructions

A.2 Live Demonstration

A live, interactive demonstration of the VizEngine is available at:

`https://www.viz-engine.com`

The live demo allows users to:

- Explore pre-built example visualizations
- Create and edit audio-reactive animations in real-time
- Experiment with the node-based animation system
- Export and share custom projects
- Test all features described in this thesis

Both resources are maintained and will continue to be developed as part of the ongoing evolution of the VizEngine platform.

Appendix B

Profiler Metric Definitions

This appendix provides brief technical definitions for the key performance metrics collected by the custom in-engine profiler, as presented in Chapter 6. The profiler uses standard browser APIs like `requestAnimationFrame` and `performance.now()` for high-precision timing.

B.1 Core Timing Metrics

FPS (Frames Per Second) Measured by counting the number of frames (n_{frames}) within a fixed time window and dividing by the elapsed time (t_e) in seconds, as shown in Equation B.1.

$$FPS = \frac{n_{frames}}{t_e} \quad (\text{B.1})$$

The mean *FPS* is the average of these windowed measurements over the session.

Frame Time (ms) The actual time elapsed between successive `requestAnimationFrame` calls, measured using `performance.now()`. This provides a precise measurement for each individual frame rendered.

- **Mean Frame Time ($\bar{t}_f$):** The arithmetic average of all individual frame times recorded during the session. Calculated as shown in Equation B.2,

where t_i is the time for frame i and N is the total number of frames.

$$\bar{t}_f = \frac{1}{N} \sum_{i=1}^N t_i \quad (\text{B.2})$$

- **Max Frame Time ($t_{f,max}$):** The single longest frame time recorded during the entire session. Calculated as shown in Equation B.3, where t_i are the individual frame times.

$$t_{f,max} = \max(t_1, t_2, \dots, t_N) \quad (\text{B.3})$$

This metric is crucial for identifying worst-case performance spikes.

Frame Budget Usage (%) An estimate of main thread utilization relative to a target frame rate (typically 60 FPS). It is calculated based on the mean frame time ($\bar{t}_f$) within a measurement window (e.g., 200ms). The budget usage (U_{budget}) is given by Equation B.4, where $\bar{t}_f$ is in milliseconds.

$$U_{budget} = \min \left(100, \frac{\bar{t}_f}{16.67} \times 100 \right) \quad (\text{B.4})$$

A value of 100% indicates the main thread is fully utilized at the target frame rate. Values over 100% (capped at 100 for reporting) indicate frames are being dropped. The profiler also incorporates data from the Long Task API, when available, to more accurately reflect main thread blocking.

Frame Stability Score A score from 0 to 100 representing the consistency of frame delivery, calculated using the Coefficient of Variation (CV) of the windowed FPS measurements recorded during the session. The CV is the ratio of the standard deviation (σ) to the mean (μ) of the FPS , as shown in Equation B.5.

$$CV = \frac{\sigma_{FPS}}{\mu_{FPS}} \quad (\text{B.5})$$

The final stability score ($S_{stability}$) is then calculated (Equation B.6).

$$S_{stability} = \max(0, 1 - CV) \times 100 \quad (\text{B.6})$$

A lower CV (less variation) results in a higher $S_{stability}$ score, with 100 representing perfect stability (zero variation in FPS).

B.2 Component-Specific Metrics

Layer Render Time (ms) The time taken for a specific layer’s rendering function (`draw` or `draw3D`) to execute within a single frame. This render time, t_{render} , is measured by recording the time before (t_{start}) and after (t_{end}) the function call, as shown in Equation B.7.

$$t_{render} = t_{end} - t_{start} \quad (\text{B.7})$$

The average ($\bar{t}_{render}$) and maximum ($t_{render,max}$) render times are calculated across all frames in the session for each layer.

Node Network Computation Time (ms) The time taken to evaluate the entire node graph associated with a specific animated parameter within a single frame. This computation time, $t_{compute}$, is measured by recording t_{start} and t_{end} before and after the recursive, memoized evaluation of the network, as shown in Equation B.8.

$$t_{compute} = t_{end} - t_{start} \quad (\text{B.8})$$

The average ($\bar{t}_{compute}$) and maximum ($t_{compute,max}$) computation times are calculated across all frames in the session for each active network.

Bibliography

- [1] Resolume, Resolume vj software, accessed: November 9, 2025 (2025).
URL <https://resolume.com/>
- [2] Derivative, Touchdesigner, accessed: October 2, 2025 (Mar 2020).
URL <https://derivative.ca/feature/application-building>
- [3] Processing, Processing, accessed: October 5, 2025 (2025).
URL <https://processing.org/>
- [4] p5.js, p5.js, accessed: October 7, 2025 (2025).
URL <https://p5js.org/>
- [5] M. Graf, H. C. Opara, M. Barthet, An audio-driven system for real-time music visualisation, arXiv preprint arXiv:2106.10134 (2021).
- [6] M. Marinko, Sound nodes - audio visualization, accessed: October 9, 2025 (2025).
URL <https://blendermarket.com/products/sound-nodes/docs>
- [7] L. R. Rabiner, R. W. Schafer, Digital processing of speech signals, Prentice-Hall, Englewood Cliffs, N.J., 1978.
- [8] M. M"uller, Fundamentals of Music Processing - Audio, Analysis, Algorithms, Applications, Springer, 2015.
- [9] J. W. Cooley, J. W. Tukey, An algorithm for the machine calculation of complex fourier series, Mathematics of Computation 19 (90) (1965) 297–301. doi:10.1090/S0025-5718-1965-0178586-1.

-
- [10] J. M. Grey, J. W. Gordon, Perceptual effects of spectral modifications on musical timbres, *The Journal of the Acoustical Society of America* 63 (5) (1978) 1493–1500. doi:10.1121/1.381843.
- [11] P. Mermelstein, Distance measures for speech recognition, psychological and instrumental, in: C. H. Chen (Ed.), *Pattern Recognition and Artificial Intelligence*, Academic Press, 1976, pp. 374–388.
- [12] S. B. Davis, P. Mermelstein, Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences, *IEEE Transactions on Acoustics, Speech, and Signal Processing* 28 (4) (1980) 357–366. doi:10.1109/TASSP.1980.1163420.
- [13] R. D. Patterson, J. Holdsworth, I. Nimmo-Smith, P. Rice, An efficient auditory filterbank based on the gammatone function, Tech. rep., MRC Applied Psychology Unit, Cambridge, UK, annex to the SVOS final report, Part A: The auditory filterbank (1987).
- [14] X. Valero, F. Alías, A computationally efficient gammatone filterbank for auditory feature extraction, in: *Proceedings of the 15th International Conference on Digital Audio Effects (DAFx-12)*, York, UK, 2012, pp. 1–4.
- [15] A. A. Correya, D. Bogdanov, L. Joglar-Ongay, X. Serra, *Essentia*, accessed: October 11, 2025 (2025).
URL <https://mtg.github.io/essentia.js/>
- [16] A. A. Correya, J. Marcos Fernández, L. Joglar-Ongay, P. Alonso Jiménez, X. Serra, D. Bogdanov, Audio and music analysis on the web using *essentia.js*, *Transactions of the International Society for Music Information Retrieval*. 2021; 4 (1): 167-81. (2021).
- [17] H. Rawlinson, N. Segal, J. Fiala, *Meyda: an audio feature extraction library for the web audio api*, in: *The 1st web audio conference (WAC)*. Paris, Fr, 2015.

-
- [18] O. Lartillot, P. Toiviainen, A matlab toolbox for musical feature extraction from audio, in: International conference on digital audio effects, Vol. 237, Bordeaux, 2007, p. 244.
 - [19] D. Bonet-Solà, F. J. Alsina-Pagès, A comparative survey of feature extraction and machine learning methods in diverse acoustic environments, *Sensors* 21 (3) (2021) 925. doi:10.3390/s21030925.
 - [20] K. W. W. Group, WebGL specification, accessed: October 21, 2025 (2011).
URL <https://www.khronos.org/registry/webgl/specs/latest/1.0/>
 - [21] Mr.doob, T. contributors, Three.js - javascript 3d library, accessed: October 13, 2025 (2025).
URL <https://threejs.org/>
 - [22] I. Meta Platforms, React, accessed: October 15, 2025 (2025).
URL <https://react.dev/>
 - [23] Notch, Notch - the real-time graphics tool, accessed: October 23, 2025 (2025).
URL <https://www.notch.one/>
 - [24] C. '74, Max msp jitter, accessed: October 25, 2025 (2025).
URL <https://cycling74.com/products/max>
 - [25] Undev, cables.gl - your visual playground for the web, accessed: October 17, 2025 (2025).
URL <https://cables.gl/>
 - [26] O. Jack, Hydra: Live-codeable video synth and coding environment, accessed: October 19, 2025 (2025).
URL <https://hydra.ojack.xyz/>

-
- [27] P. Hudak, Conception, evolution, and application of functional programming languages, *ACM Computing Surveys (CSUR)* 21 (3) (1989) 359–411. doi:10.1145/72551.72554.
 - [28] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1995.
 - [29] A. Sorensen, A. Tarabella, The impromptu node-based interface for live coding, in: *Proceedings of the International Conference on New Interfaces for Musical Expression*, Sydney, Australia, 2010, pp. 243–246.
 - [30] Google, *Firebase: Build and run successful apps*, accessed: 2025 (2025). URL <https://firebase.google.com/>
 - [31] Supabase Inc., *Supabase: The open source firebase alternative*, accessed: 2025 (2025). URL <https://supabase.com/>
 - [32] Convex Inc., *Convex: The backend-as-a-service for real-time applications*, accessed: 2025 (2025). URL <https://convex.dev/>
 - [33] T. Porter, T. Duff, Compositing digital images, *Computer Graphics (ACM SIGGRAPH '84 Proceedings)* 18 (3) (1984) 253–259. doi:10.1145/800031.808606.
 - [34] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, D. Holm, T. Wagner, J. Gloger, J.-F. Bastien, A. Pop, A. Zakai, Bringing webassembly to the web, in: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200. doi:10.1145/3062341.3062363.
 - [35] Vercel, *Next.js by vercel - the react framework for the web*, accessed: 2025 (2025). URL <https://nextjs.org/>

-
- [36] shadcn, Shadcn/ui, accessed: 2025 (2025).
URL <https://ui.shadcn.com/>
- [37] XYFlow Team, React flow - a library for building interactive node-based uis, formerly known as React Flow, maintained by XYFlow. Accessed: 2025 (2025).
URL <https://reactflow.dev/>
- [38] Cloudflare Inc., dnd kit - a modern, lightweight, performant, accessible and extensible drag & drop toolkit for react, accessed: 2025 (2025).
URL <https://dndkit.com/>
- [39] katspaugh and contributors, Wavesurfer.js - navigable waveform audio player, accessed: 2025 (2025).
URL <https://wavesurfer.xyz/>
- [40] J. Wu, Ffmpeg.wasm: Ffmpeg for browser and node, <https://github.com/ffmpegwasm/ffmpeg.wasm>, accessed: October 31, 2025 (2020).
- [41] A. Guerchouche, F. Duguet, R. Castagné, Gpu-based animation of large crowds of non-humanoid characters, in: Proceedings of the 18th International Conference on 3D Web Technology, Web3D '13, ACM, New York, NY, USA, 2013, pp. 121–128. doi:10.1145/2466533.2466555.
- [42] Í. Quílez, Rendering worlds with two triangles with raytracing on the GPU in 4k, Online Article, accessed: October 27, 2025 (2008).
URL <https://iquilezles.org/articles/raymatchingdf/>
- [43] B. Karis, Real shading in unreal engine 4, Physically Based Shading in Theory and Practice, SIGGRAPH 2013 Course Notes, accessed: October 29, 2025 (2013).
URL <https://cdn2.unrealengine.com/Resources/files/2013SiggraphPresentationsNotes-26915738.pdf>